

KIP Technical Documentation

Technical Architecture & Handover Documentation

17 July 2026

About this document

1. Executive Summary

2. Backend Deep Dive — Core Infrastructure

backend/main.py
backend/app/database.py
backend/app/security.py
backend/app/schemas.py
Request Lifecycle

3. Backend Deep Dive — Data Models

business_chat_model.py
business_dashboard.py
business_idea.py
conversation.py
enhanced_logs.py
enterprise_models.py
models_briefing.py
news.py
startup_chat.py
subscription_models.py
user.py
Entity-Relationship Narrative
Static Reference Data (backend/app/data/)

4. Backend Deep Dive — Auth, Chat & Core Routes

auth.py
auth-stripe.py
chat.py
dashboard.py
ideas.py
language_support.py
i18n_routes.py
startup_chat.py
add_own_business.py
briefings.py

5. Backend Deep Dive — Business Operations Routes

business_chat.py
business_dashboard.py
capital_access.py
enhanced_logs.py

6. Backend Deep Dive — Enterprise, News & Templates Routes

enterprise.py
news.py
templates.py

7. Backend Deep Dive — Learning & BizSim Game

learn.py — KIP Learn (financial literacy module)
bizsim_engine.py — BizSim simulation logic
bizsim_routes.py — BizSim API endpoints

8. Backend Deep Dive — The AI Engine (services/)

kip_engine.py

kip_prompt.py
kip_score.py
knowledge_base.py
ml_engine.py
news_service.py
AI Pipeline — end-to-end narrative

9. Backend Deep Dive — Supporting Services

briefing_service.py
business_plan_generator.py
dashboard_service.py
email_service.py
log_service.py
map_service.py
subscription_service.py
survey_json.py
zedtranslate_service.py

10. Frontend Architecture

Routing & App shell
Auth flow
API layer
State & context patterns
Page inventory
Component inventory
Notable patterns and issues

11. Challenges, Vulnerabilities & Solutions

1. Secrets management
2. Authentication & JWT
3. CORS
4. Injection risks
5. Payment security
6. File uploads / path traversal
7. Rate limiting & Claude API cost control
8. Dependencies & deployment
9. Cross-site scripting (XSS)
10. Untracked / backup files

Appendix A — Consolidated Punch List

Do first (security / cost / broken-in-production)
Do soon (correctness bugs, data integrity)
Worth scheduling (tech debt, consistency, performance)

Appendix B — Glossary

TECHNICAL DOCUMENTATION

KIP

Kwacha Intelligence Platform

Technical Architecture & Engineering Handover Guide

Prepared: 17 July 2026

Audience: Incoming engineers, technical leads, and IT staff taking over or extending the KIP codebase

Scope: Full backend (FastAPI/Python, 57 files, ~12,300 lines) and frontend (React/Vite, 58 files) source walkthrough, architecture, data model, and a security/technical-debt audit

About this document

This document exists so that a new engineer — with general software experience but no prior exposure to this codebase — can become productive on KIP without needing a live handover conversation. It is organized in the order a new hire should read it:

1. **Executive Summary & Architecture** — what KIP is, how the pieces fit together, and the request lifecycle.
2. **Backend Deep Dive** — every Python file in `backend/app/`, explained block by block, with exact file and line-number references (`file.py:12-34`) so you can jump straight to the source.
3. **Frontend Architecture** — how the React app is structured, its routing, auth, and API layers.
4. **Challenges, Vulnerabilities & Solutions** — a candid, evidence-based audit of what's fragile, risky, or unfinished in the current system, each with a concrete recommended fix.
5. **Appendix** — a consolidated punch-list of every issue found in this review, ranked by severity, plus a glossary of terms for readers newer to this stack.

How to read the code references. Every claim in this document that describes actual code is cited as `path/to/file.py:START-END`, matching the line numbers in the codebase as of the commit reviewed (`cc47629`, 17 July 2026). If the code has since changed, use the cited line range as a starting point to locate the (possibly moved) logic rather than as a guarantee it's unchanged.

A note on tone. This document does not sugar-coat problems. Sections flagged **Note:** throughout the backend/frontend walkthroughs, and the entire "Challenges, Vulnerabilities & Solutions" chapter, describe real gaps found by reading the actual source — missing validation, dead code, silent failures, and a few active bugs. This is intentional: the goal is for you to inherit an accurate mental model of the system, not a polished one.

1. Executive Summary

KIP (Kwacha Intelligence Platform) is an AI-powered business-advisory web application for Zambian entrepreneurs. A user chats with an AI advisor (“K-BIG-1”) that is grounded in Zambian economic data (loan sources, registration costs, tax rules, town-level market intelligence) via retrieval-augmented generation (RAG), and can turn a generated idea into a live “business dashboard” — a launch plan, daily operating logs, ML-based revenue forecasting, AI coaching, and (as a gamified companion) a full business-simulation game (“BizSim”). The platform also ships a financial-literacy course module, a multi-branch “enterprise” feature for larger operators, a capital-access/funding-matcher, and localized UI/AI-content translation into Zambian languages.

Stack at a glance:

Layer	Technology	Notes
Frontend	React 18 + Vite, React Router v6, Tailwind CSS	SPA, deployed to Vercel (<code>frontend/vercel.json</code>)
Backend	FastAPI (Python), Uvicorn	Deployed to Fly.io (<code>backend/Dockerfile</code> , <code>backend/fly.toml</code>)
Database	SQLAlchemy ORM over SQLite (dev) / any SQL DB via <code>DATABASE_URL</code> (prod)	No formal migration tool (Alembic) in use — see Chapter 4
AI	Anthropic Claude (<code>claude-sonnet-4-6</code>), called directly via the <code>anthropic</code> Python SDK	Used for chat, idea generation, coaching, PDF plan content, BizSim NPCs, translation review
RAG	ChromaDB (persistent, local/volume-mounted) + <code>sentence-transformers/all-MiniLM-L6-v2</code> embeddings	Grounds AI answers in a curated Zambian business/economics knowledge base
Auth	Stateless JWT (HS256, 7-day expiry), bcrypt password hashing	No refresh tokens or revocation — see Chapter 6
External data	OpenStreetMap Overpass API (local business density), open.er-api.com (USD/ZMW), Zambian news RSS feeds, ZedTranslate API (language translation)	Several of these have caching layers; some are stubbed/broken (see Chapter 6)

How the codebase is organized:

backend/	
main.py	entrypoint: creates the FastAPI app, wires up every router
app/	
database.py	SQLAlchemy engine/session setup
security.py	JWT + password hashing, the <code>get_current_user</code> dependency
schemas.py	Pydantic request/response models
models/	SQLAlchemy ORM tables (one file per feature area)
routes/	FastAPI routers — one file per feature, mounted under <code>/api/*</code>
services/	business logic & AI calls, imported by routes
data/	static reference data (Zambian towns, seed knowledge, coordinates)
frontend/	
src/	
main.jsx, App.jsx	app bootstrap and router
pages/	one component per route/screen
components/	shared/reusable UI, including the BizSim game UI
hooks/, context/	auth, theme, language, offline-queue state
lib/	the shared axios API client and offline sync helpers

Request lifecycle (backend): a browser request hits FastAPI’s CORS middleware, is routed to the matching `APIRouter` (mounted under an `/api/...` prefix in `main.py`), resolves its declared dependencies (most commonly `get_current_user`, which verifies the JWT and loads the SQLAlchemy `User`, and `get_db`, which opens/closes a database session), runs the endpoint’s business logic (often delegating to a `services/*.py` function, which may call the Anthropic API), and returns a response serialized through a Pydantic schema. The full mechanics of this — with exact line citations — are in Chapter 2.

What's genuinely solid: the JWT auth dependency pattern is used consistently across almost every endpoint; SQL is parameterized everywhere (no SQL-injection vectors were found in this review); CORS is explicitly origin-scoped rather than wildcarded; the RAG/AI pipeline degrades gracefully (missing API key or a failed call falls back to canned text rather than crashing); and the email service never hardcodes credentials.

What needs attention before this system scales or takes payments again: there is currently **no cost control** on any Claude-calling endpoint (the rate-limiting code exists in `subscription_service.py` but is wired into nothing); the **entire payment backend (Stripe/Flutterwave) was deleted in the most recent commit** while the frontend still calls it; the JWT signing key has an insecure hardcoded fallback; and several features (BizSim's debt mechanic, `/day/run`'s input validation, the `/survey` update endpoint) contain active bugs found during this review. All of this is detailed with evidence in Chapter 6 and summarized as a punch-list in the Appendix.

2. Backend Deep Dive — Core Infrastructure

This chapter covers the four files that every request passes through regardless of which feature it touches: the app entrypoint, the database session machinery, the authentication system, and the shared Pydantic schemas.

backend/main.py

What this file is for: This is the FastAPI application entrypoint — the single file that boots the whole backend. It loads environment variables, imports every SQLAlchemy model so their tables get registered, creates any missing database tables, runs a small ad-hoc migration for one table, constructs the `FastAPI` app, configures CORS, wires up every feature area as a router under an `/api/...` prefix, and exposes two trivial health-check endpoints. Running `uvicorn main:app` starts the entire KIP API from this file.

Imports and environment loading — `main.py:1-6`

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from dotenv import load_dotenv
import os

load_dotenv()
```

`load_dotenv()` reads a `.env` file (if present) into `os.environ` **before** anything else in the file runs. This must happen before `app.database` and other modules are imported, because those modules read `os.getenv(...)` at import time (e.g. `DATABASE_URL`, `SECRET_KEY`). Import order in this file is therefore load-bearing, not cosmetic.

Model imports — `main.py:8-23`

```
from app.database import engine, Base
from app.models import user, conversation
from app.models import business_dashboard
...
try:
    from app.models import enterprise_models
except ImportError:
    pass
try:
    from app.models import subscription_models
except ImportError:
    pass
```

SQLAlchemy's declarative `Base` only knows about a model class once its module has been imported (importing registers the table on `Base.metadata`). This block force-imports every model module purely for that side effect — none of the imported names besides `engine` / `Base` are used directly in this file. The two `try/except ImportError` blocks silently tolerate `enterprise_models` or `subscription_models` not existing, meaning those tables simply won't be created if the modules are missing, with no warning printed.

Note: swallowing `ImportError` silently here can mask a genuine typo/missing dependency, not just an intentionally-optional module — a broken import would fail the same way a legitimately-absent module does.

Route module imports — `main.py:25-41`

```
from app.routes import (
    auth, chat, ideas, dashboard, news,
    business_dashboard as bd_routes,
    ...
)
from app.routes import enterprise
from app.routes import add_own_business
...
from app.routes.briefings import router as briefings_router
from app.routes import i18n_routes
```

Every feature router is imported here so it can later be attached with `app.include_router(...)`. Note the aliasing: the `business_dashboard` route module is imported as `bd_routes` specifically to avoid a name clash with the `business_dashboard` model module imported a few lines above.

Table creation — `main.py:45`

```
Base.metadata.create_all(bind=engine)
```

Creates any tables that don't already exist, for every model class registered on `Base` (via the imports above). This is a lightweight substitute for a real migration tool (e.g. Alembic) — it only ever adds missing tables, it never alters or drops columns on existing tables.

Ad-hoc schema migration — `main.py:47-73`

```
def _migrate_user_verification_columns():
    """Add email-verification columns to an existing `users` table created before they existed."""
    ...
    if "is_verified" not in existing:
        statements.append(f"ALTER TABLE users ADD COLUMN is_verified {bool_type}")
    ...
try:
    _migrate_user_verification_columns()
except Exception:
    pass
```

Because `create_all` never modifies existing tables, this function hand-rolls a minimal migration: it inspects the live `users` table, and for each of three verification-related columns that's missing, issues a raw `ALTER TABLE` statement. It branches on `engine.dialect.name` to pick SQLite vs. Postgres boolean-default syntax. The whole call is wrapped in a bare `try/except Exception: pass` — **Note:** any failure here (e.g. a locked SQLite file, a permissions issue, or a genuinely broken `ALTER TABLE`) is silently swallowed, so the app will keep starting even if the migration didn't actually apply, potentially causing confusing `column does not exist` errors later at request time.

App construction and CORS — `main.py:75-89`

```
app = FastAPI(title="Kwacha Intelligence Platform – KIP API", version="4.0.0")

FRONTEND_URLS = [
    "http://localhost:3000",
    "http://localhost:5173",
    os.getenv("FRONTEND_URL", ""),
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=[u for u in FRONTEND_URLS if u],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Two hard-coded local dev origins (Create-React-App on 3000, Vite on 5173) plus one production origin read from the `FRONTEND_URL` env var. The list comprehension `[u for u in FRONTEND_URLS if u]` filters out the empty string so an unset `FRONTEND_URL` doesn't register `""` as an allowed origin. `allow_credentials=True` is required because the frontend sends the JWT bearer token; combined with `allow_methods=["*"]` / `allow_headers=["*"]` this is a permissive-but-scoped CORS policy (scoped by explicit origin list, not wildcarded origins — browsers disallow `allow_origins=["*"]` together with `allow_credentials=True` anyway).

Router registration — `main.py:91-108`

```
app.include_router(auth.router, prefix="/api/auth", tags=["Auth"])
...
app.include_router(briefings_router, prefix="/api/news", tags=["briefings"])
```

See the router table below. Each call mounts a feature's `APIRouter` under a URL prefix and a Swagger/OpenAPI tag for docs grouping.

Root and health endpoints — `main.py:110-116`

```
@app.get("/")
def root():
    return {"platform": "KIP", "version": "4.0.0", "status": "operational"}

@app.get("/health")
def health():
    return {"status": "healthy"}
```

Two unauthenticated, dependency-free endpoints. `/` is a friendly landing response; `/health` is the conventional endpoint used by load balancers/uptime monitors to check the process is alive. Neither touches the database, so a healthy response doesn't guarantee the DB is reachable.

Router registration table

Router module	URL prefix	Purpose
<code>app.routes.auth</code>	<code>/api/auth</code>	User registration, login, email verification, JWT issuance
<code>app.routes.i18n_routes</code>	<code>/api/i18n</code>	Serves UI translation strings, on-demand string/AI-content translation, saves user language preference
<code>app.routes.chat</code>	<code>/api/chat</code>	Main KIP AI chat/conversation flow (idea generation via <code>kip_engine</code>)

Router module	URL prefix	Purpose
<code>app.routes.ideas</code>	<code>/api/ideas</code>	Fetching and giving feedback (accept/decline) on saved business ideas
<code>app.routes.dashboard</code>	<code>/api/dashboard</code>	Aggregate dashboard stats (conversation/idea counts, KIP score)
<code>app.routes.news</code>	<code>/api/news</code>	News scraping, USD/ZMW rates, fuel prices, alert generation
<code>app.routes.business_dashboard</code> (aliased <code>bd_routes</code>)	<code>/api/business</code>	Business launch plans and daily business logs (start plan, log entries)
<code>app.routes.enhanced_logs</code>	<code>/api/logs</code>	Daily logs, ML predictions, weekly reviews, market survey
<code>app.routes.templates</code>	<code>/api/templates</code>	Document/survey templates (e.g. general survey with numeric coercion)
<code>app.routes.business_chat</code>	<code>/api/business-chat</code>	Chat flow scoped to an existing/own business (raw-SQL backed tables)
<code>app.routes.startup_chat</code>	<code>/api/startup-chat</code>	Chat flow scoped to startup-stage conversations
<code>app.routes.enterprise</code>	<code>/api/enterprise</code>	Enterprise creation, branch management, consolidated dashboard, branch notifications
<code>app.routes.add_own_business</code>	<code>/api/business</code>	Lets users with an existing business add it and get a growth-focused plan
<code>app.routes.capital_access</code>	<code>/api/capital</code>	Matches business profile to Zambian funding sources with eligibility + AI guidance
<code>app.routes.learn</code>	<code>/api/learn</code>	Financial-literacy courses, lessons, quizzes, progress, badges
<code>app.routes.bizsim_routes</code>	<code>/api/bizsim</code>	"BizSim" business simulation game (sessions, day runs, statements, loans, leaderboard)
<code>app.routes.briefings</code> (<code>briefings_router</code>)	<code>/api/news</code>	Cached daily news briefings + history + manual refresh (shares prefix with <code>news</code>)

Note: `bd_routes` (business_dashboard) and `add_own_business` both mount under `/api/business` , and `briefings` shares `/api/news` with the `news` router. This is not a bug — their individual route paths don't collide (e.g. `/api/business/add-own` vs. `/api/business/start`) — but it means the "one router = one prefix" mental model doesn't hold everywhere, and a future path added to one could silently collide with the other. Worth being deliberate about when adding new endpoints to either module.

backend/app/database.py

What this file is for: This is the single source of truth for how KIP talks to its SQL database. It creates the SQLAlchemy `engine` , the `SessionLocal` session factory, the declarative `Base` class every model inherits from, and the `get_db` dependency that request handlers use to obtain a database session. Every other backend module that touches the database imports from here.

Imports and DB URL — `database.py:1-6`

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
import os

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./kip.db")
```

The database location is entirely env-driven, defaulting to a local SQLite file `kip.db` in the working directory if `DATABASE_URL` isn't set. This makes local development zero-config while production can point at Postgres/MySQL/etc. via env var.

SQLite connect args — `database.py:8-9`

```
# SQLite needs this setting; PostgreSQL does not
connect_args = {"check_same_thread": False} if DATABASE_URL.startswith("sqlite") else {}
```

SQLite connections are by default only usable from the thread that created them. FastAPI's sync route handlers can run on different threads (via its threadpool), so `check_same_thread=False` disables that restriction. This flag is SQLite-specific and is a no-op/absent for other databases (hence the conditional).

Engine, session factory, declarative base — `database.py:11-13`

```
engine = create_engine(DATABASE_URL, connect_args=connect_args)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
```

- `engine` is the DBAPI connection pool used throughout the app (also imported directly by `main.py` for `create_all` and the migration helper).
- `SessionLocal` is a session **factory** (not a session itself) — calling `SessionLocal()` produces a new `Session` object. `autocommit=False` means changes require an explicit `commit()`; `autoflush=False` means pending changes aren't automatically flushed to the DB before every query (giving handlers explicit control over when writes happen).
- `Base` is the declarative base class every ORM model (in `app/models/*`) subclasses; `Base.metadata` is what `main.py` calls `create_all` on.

Note: `declarative_base` is imported from `sqlalchemy.ext.declarative`, which is the deprecated location in modern SQLAlchemy (2.x prefers `sqlalchemy.orm.declarative_base`). It still works but emits a deprecation warning depending on installed SQLAlchemy version.

`get_db` dependency — `database.py:15-21`

```
def get_db():
    """Dependency: yields a database session, closes it when done."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

This is a FastAPI dependency using the generator pattern: FastAPI calls `get_db()`, gets an iterator, advances it once (running code up to `yield`) to obtain `db`, injects that into the route handler, and after the handler returns (or raises), resumes the generator past the `yield` so the `finally` block runs and closes the session. This guarantees the session is closed exactly once per request regardless of whether the handler succeeded or threw an exception — a critical resource-cleanup pattern so DB connections aren't leaked under load or on errors.

backend/app/security.py

What this file is for: Centralizes all authentication logic: password hashing/verification with `bcrypt`, and JWT creation/validation for session tokens. It also defines `get_current_user`, the FastAPI dependency that essentially every protected route in KIP depends on to identify "who is making this request."

Imports and configuration constants — `security.py:1-16`

```
import jwt
import bcrypt

from datetime import datetime, timedelta, timezone
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from sqlalchemy.orm import Session

import os

from app.database import get_db
from app.models.user import User

SECRET_KEY = os.getenv("SECRET_KEY", "kip-dev-secret-change-in-production")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_HOURS = 24 * 7  # 7 days

bearer_scheme = HTTPBearer()
```

- `SECRET_KEY` falls back to a hard-coded dev value if the `SECRET_KEY` env var isn't set. **Note:** this is a real security risk if `SECRET_KEY` is accidentally left unset in a production deployment — anyone would be able to forge valid JWTs using the publicly-visible default string in this source file.
- `ALGORITHM = "HS256"` is a symmetric-key JWT signing algorithm (same key signs and verifies), simple and fast, appropriate for a single-backend service (no need for asymmetric keys since there's no separate verifying party).
- `ACCESS_TOKEN_EXPIRE_HOURS = 24 * 7` (7 days) is a long-lived token — a UX tradeoff favoring "stay logged in for a week" over frequent re-authentication, at the cost of a stolen token remaining valid for up to a week (there's no visible refresh-token or revocation/blacklist mechanism in this file).
- `bearer_scheme = HTTPBearer()` is FastAPI's built-in security scheme for reading an `Authorization: Bearer <token>` header; it also makes protected endpoints show a padlock/auth requirement in the Swagger UI.

`hash_password / verify_password` — `security.py:18-24`

```
def hash_password(password: str) -> str:
    """Hash a plain-text password using bcrypt."""
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

def verify_password(plain: str, hashed: str) -> bool:
    """Check a plain-text password against a stored bcrypt hash."""
    return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("utf-8"))
```

`bcrypt.gensalt()` generates a fresh random salt for every call (with no fixed cost factor argument, so it uses bcrypt's library default, typically cost factor 12), meaning two users with the same password get different hashes — this defends against rainbow-table attacks and makes brute-forcing computationally expensive by design (bcrypt is deliberately slow, unlike a fast hash such as SHA-256). `checkpw` re-derives the hash using the salt embedded in the stored hash string and compares in constant time, avoiding timing-attack leakage.

`create_access_token` — `security.py:26-30`

```
def create_access_token(user_id: int) -> str:
    """Create a signed JWT token for the given user ID."""
    expire = datetime.now(timezone.utc) + timedelta(hours=ACCESS_TOKEN_EXPIRE_HOURS)
    payload = {"sub": str(user_id), "exp": expire}
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)
```

The JWT payload is intentionally minimal — just the standard `sub` (subject, here the user ID as a string per JWT convention) and `exp` (expiry) claims. `datetime.now(timezone.utc)` is used (not naive `datetime.now()`) so expiry comparisons are unambiguous regardless of server timezone. No other user data (email, role, etc.) is embedded in the token, so any authorization decision beyond "is this a valid user" requires a DB lookup — which is exactly what `get_current_user` does next.

get_current_user dependency — security.py:32-49

```
def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme),
    db: Session = Depends(get_db)
) -> User:
    """Dependency: decodes JWT and returns the logged-in user."""
    token = credentials.credentials

    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id = int(payload.get("sub"))
    except (jwt.ExpiredSignatureError, jwt.InvalidTokenError, ValueError):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or expired session. Please log in again.",
        )

    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="User not found.")
    return user
```

This is a **nested dependency**: it itself depends on `bearer_scheme` (to extract the token from the `Authorization` header) and `get_db` (to query the database), and FastAPI resolves both before calling this function. `jwt.decode` both verifies the signature (using `SECRET_KEY`) and checks the `exp` claim automatically, raising `ExpiredSignatureError` or `InvalidTokenError` as appropriate; `ValueError` is also caught in case `payload.get("sub")` is missing/non-numeric and `int(...)` fails. Any of those failure modes collapse into a uniform `401 Unauthorized`. If the token is valid but the referenced user no longer exists in the DB (e.g. account deleted), a `404` is raised instead — **Note**: this is a slightly unusual choice; a `401` might be more conventional here since it's fundamentally an authentication failure, and returning `404` on an auth dependency could be confusing to a frontend expecting only 401s from this dependency. Any route that declares `current_user: User = Depends(get_current_user)` automatically gets this whole chain (bearer extraction → JWT verification → DB lookup) run before the handler body executes.

backend/app/schemas.py

What this file is for: Defines all the Pydantic models used as FastAPI request bodies and response shapes across the auth, chat, startup-chat, business-ideas, and dashboard features. These schemas are what generates KIP's OpenAPI/Swagger docs and what performs automatic request validation and response serialization/filtering (stripping out any ORM fields not listed in the schema).

Imports — schemas.py:1-3

```
from pydantic import BaseModel, EmailStr
from typing import Optional, List
from datetime import datetime
```

`EmailStr` requires the `email-validator` package to be installed; it validates that a string is a syntactically well-formed email address at request-parsing time, rejecting malformed input before it ever reaches a route handler.

```
class UserRegister(BaseModel):
    full_name: str
    email: EmailStr
    password: str

class UserLogin(BaseModel):
    email: EmailStr
    password: str

class UserOut(BaseModel):
    id: int
    full_name: str
    email: str
    plan_tier: str
    is_verified: bool
    created_at: datetime

    class Config:
        from_attributes = True

class Token(BaseModel):
    access_token: str
    token_type: str
    user: UserOut
```

`UserRegister` / `UserLogin` are pure input schemas (no `Config`, since they're never constructed from an ORM object). `UserOut` is an output schema with `from_attributes = True` (Pydantic v2's replacement for the old `orm_mode`), which lets it be built directly from a SQLAlchemy `User` ORM instance's attributes (`UserOut.model_validate(user_obj)` or via FastAPI's `response_model`) rather than requiring a dict. Critically, `UserOut` does **not** include the password hash field — this is the mechanism that prevents password hashes from ever leaking into an API response, since `response_model` filtering only serializes fields declared on the schema. `Token` is the login/register response envelope: JWT string, token type (conventionally `"bearer"`), and the nested `UserOut` .

Chat schemas — `schemas.py:30-54`

```
class ChatMessage(BaseModel):
    message: str
    conversation_id: Optional[int] = None

class MessageOut(BaseModel):
    id: int
    role: str
    content: str
    created_at: datetime
    class Config:
        from_attributes = True

class ConversationOut(BaseModel):
    id: int
    title: str
    created_at: datetime
    messages: List[MessageOut] = []
    class Config:
        from_attributes = True

class ChatResponse(BaseModel):
    conversation_id: int
    reply: str
    idea_saved: bool = False
```

`ChatMessage.conversation_id` being `Optional[int] = None` lets the same endpoint handle both “start a new conversation” (omit the field) and “continue an existing one” (pass its ID) — the route handler branches on whether it’s `None`. `ConversationOut` nests `List[MessageOut]`, so returning a `ConversationOut` from a route automatically serializes the full message history in one response. `ChatResponse.idea_saved` defaults to `False`, signaling to the frontend whether this particular chat turn resulted in a business idea being persisted (a KIP-specific behavior where the AI chat can trigger a side-effect save).

Startup Chat schemas — `schemas.py:56-81`

```
class StartupChatMessage(BaseModel):
    message: str
    conversation_id: Optional[int] = None
    step_context: Optional[str] = None

class StartupMessageOut(BaseModel):
    ...

class StartupConversationOut(BaseModel):
    id: int
    title: str
    step_emoji: Optional[str] = None
    created_at: datetime
    messages: List[StartupMessageOut] = []
    class Config:
        from_attributes = True

class StartupChatResponse(BaseModel):
    conversation_id: int
    reply: str
```

This is a parallel, near-duplicate schema set to the general Chat schemas above, dedicated to the “startup chat” flow (`app.routes.startup_chat`). The distinguishing fields are `step_context` (input, tells the AI which onboarding step the user is on) and `step_emoji` (output, a UI decoration per conversation). **Note:** the duplication between `ChatMessage` / `StartupChatMessage` and

`MessageOut` / `StartupMessageOut` etc. is a maintenance smell — the two schema families are structurally near-identical except for the extra step fields; a shared base class could reduce duplication, though keeping them separate does allow the two flows to evolve independently.

Business Ideas schemas — `schemas.py:83-97`

```
class IdeaOut(BaseModel):
    id: int
    idea_name: str
    idea_summary: str
    location: Optional[str]
    capital_amount: Optional[float]
    accepted: Optional[bool]
    created_at: datetime

    class Config:
        from_attributes = True

class IdeaFeedback(BaseModel):
    accepted: bool
    decline_reason: Optional[str] = None
```

`IdeaOut.accepted` is `Optional[bool]` (three-state: `True` / `False` / `None`), modeling “not yet reviewed by the user” as `None` distinct from an explicit decline (`False`). `IdeaFeedback` is the input schema for the accept/decline endpoint; `decline_reason` is optional free text collected only when declining. **Note:** unlike other `Optional` fields in this file, `location`, `capital_amount`, and `accepted` here have no `= None` default — since this is an output-only schema (built via `from_attributes` from an ORM row) that omission is harmless, but it does mean these fields can’t be omitted if someone tried to construct `IdeaOut` directly by keyword.

Dashboard schema — `schemas.py:99-105`

```
class DashboardStats(BaseModel):
    total_conversations: int
    total_ideas: int
    accepted_ideas: int
    member_since: datetime
```

A flat aggregate-stats response used by `app.routes.dashboard`’s `/stats` endpoint — no ORM `Config` needed because the route handler constructs this directly from computed values rather than from a single ORM object.

Request Lifecycle

A browser request first hits Starlette/FastAPI’s ASGI stack, which runs registered middleware before any route code. The `CORSMiddleware` added at `main.py:83-89` inspects the `Origin` header: for a cross-origin request (e.g. the React dev server on `localhost:5173`) it checks the origin against `FRONTEND_URLS` (`main.py:77-81`); for a preflight `OPTIONS` request it short-circuits and returns the allowed methods/headers/credentials response directly, otherwise it lets the request through and adds `Access-Control-Allow-*` headers to the eventual response.

FastAPI then matches the request path against the routers mounted in `main.py:91-108` (e.g. `POST /api/chat/...` resolves into the `chat` router mounted at prefix `/api/chat`). Inside that router’s handler function, FastAPI resolves declared dependencies before running the handler body. If the handler declares `current_user: User = Depends(get_current_user)` (`security.py:32`), FastAPI first resolves `get_current_user`’s own dependencies: `bearer_scheme` (`security.py:16`) extracts and validates the presence of an `Authorization: Bearer <token>` header, and `get_db` (`database.py:15`) opens a new SQLAlchemy session by calling `SessionLocal()` and yielding it. `get_current_user` then decodes the JWT with `jwt.decode` (`security.py:39`), looks up the user by ID via the yielded `db` session (`security.py:46`), and returns the `User` ORM object, which FastAPI injects into the handler.

The handler runs its business logic against `db`, and returns either an ORM object or a dict, which FastAPI serializes through the endpoint’s `response_model` (a `schemas.py` class) — filtering to only declared fields. As the response is being sent, FastAPI resumes the `get_db` generator past its `yield` (`database.py:19`), running the `finally: db.close()` block (`database.py:20-21`) to release the session back to the pool. The CORS middleware then adds its headers on the way out, and the response reaches the browser.

3. Backend Deep Dive — Data Models

This chapter documents every SQLAlchemy table in `backend/app/models/`, followed by the three static reference-data files in `backend/app/data/`.

business_chat_model.py

What this file models: Per-business-plan chat persistence — the conversation a user has with KIP specifically about an active `BusinessLaunchPlan` (as opposed to the general-purpose `Conversation` / `Message` chat), plus a version-history mechanism for the plan's JSON so edits proposed via chat can be rolled back.

- `BusinessChatMessage` — `business_chat_model.py:10-21` (table `business_chat_messages`) Columns: `id` PK; `plan_id` Integer FK→ `business_launch_plans.id` (NOT NULL, indexed); `user_id` Integer FK→ `users.id` (NOT NULL); `role` String(20) NOT NULL; `content` Text NOT NULL; `is_plan_update_proposal` Boolean default False; `is_plan_update_applied` Boolean default False; `created_at` DateTime default utcnow. Represents one chat turn (user or KIP) tied to a specific business plan, with flags tracking whether that message proposed a plan change and whether the user accepted it. Connects to `BusinessLaunchPlan` (`business_dashboard.py`) and `User`. **Note:** `user_id` has no `index=True` despite being a FK frequently filtered on, unlike `plan_id`. No `relationship()` declared on either side (plan/user), so this table is only navigable via raw FK queries, not ORM relationships.
- `BusinessPlanVersion` — `business_chat_model.py:24-36` (table `business_plan_versions`) Columns: `id` PK; `plan_id` Integer FK→ `business_launch_plans.id` NOT NULL (not indexed); `user_id` FK→ `users.id` NOT NULL; `version_num` Integer default 1; `plan_json_old` Text NOT NULL (snapshot); `change_summary` String(500) nullable; `created_at` DateTime default utcnow. Represents a historical snapshot of a plan's JSON before a KIP-proposed change was applied, enabling revert/undo. Linked to `BusinessLaunchPlan` and `User` via FK only. **Note:** `plan_id` lacks `index=True` here (inconsistent with `BusinessChatMessage.plan_id` which is indexed) despite being the primary lookup key for "get version history for this plan." No relationships declared.

business_dashboard.py

What this file models: The core "business launch and operate" domain — a user's launched business plan, its daily operational logs, and KIP's coaching commentary generated from those logs. This is the central operational table set referenced by many other files (`business_chat_model`, `enhanced_logs`, `enterprise_models` all FK into `business_launch_plans`).

- `BusinessLaunchPlan` — `business_dashboard.py:5-22` (table `business_launch_plans`) Columns: `id` PK; `user_id` FK→ `users.id` NOT NULL; `idea_id` FK→ `business_ideas.id` nullable; `business_name` String(255) NOT NULL; `location` String(255) nullable; `capital_amount` Float nullable; `plan_json` Text NOT NULL (full plan as JSON string); `projected_monthly_profit` Float nullable; `projected_break_even_days` Integer nullable; `status` String(30) default "pre_launch"; `is_profitable` Boolean default False; `profitable_at` DateTime nullable; `created_at` / `updated_at` DateTime (onupdate=utcnow). Represents a concrete business a user has decided to launch, generated from (optionally) a `BusinessIdea`. This is the hub table that `BusinessChatMessage`, `BusinessPlanVersion`, `DailyBusinessLog`, `CoachingEntry`, all `enhanced_logs.py` tables, and `EnterpriseBranch` all hang off of. **Note:** No `relationship()` declarations anywhere in this file (pure FK-only design), and `user_id` / `idea_id` are not indexed despite being FK lookup columns.
- `DailyBusinessLog` — `business_dashboard.py:25-42` (table `daily_business_logs`) Columns: `id` PK; `plan_id` Integer FK→ `business_launch_plans.id` NOT NULL; `user_id` FK→ `users.id` NOT NULL; `log_date` DateTime default utcnow; `revenue_today` / `expenses_today` Float default 0.0; `customer_count` Integer default 0; `stock_status` String(20) default "sufficient"; `problem_today` / `highlight_today` Text nullable; `coaching_response` Text nullable; `coaching_generated_at` DateTime nullable; `created_at` DateTime default utcnow. Represents a single day's basic operating snapshot for a launched business (the simpler predecessor to `EnhancedDailyLog` in `enhanced_logs.py`). **Note:** Appears to be an earlier/simpler log schema superseded by `EnhancedDailyLog` (5-category log) — both exist and both FK to `business_launch_plans`, suggesting duplicate/legacy functionality that future engineers should verify is still in active use. Neither `plan_id` nor `user_id` is indexed.
- `CoachingEntry` — `business_dashboard.py:45-54` (table `coaching_entries`) Columns: `id` PK; `plan_id` Integer FK→ `business_launch_plans.id` NOT NULL; `user_id` FK→ `users.id` NOT NULL; `log_id` FK→ `daily_business_logs.id` nullable; `entry_type` String(30) default "daily"; `content` Text NOT NULL; `day_number` Integer nullable; `created_at` DateTime default utcnow. Represents a discrete piece of KIP coaching advice (daily/weekly/milestone/graduation) tied to a plan and optionally a specific log entry. **Note:** No indexes on any FK column.

business_idea.py

What this file models: The repository of business ideas KIP generates (or pre-loads) for a user, prior to a user turning one into an actual launch plan.

- **BusinessIdea** — `business_idea.py:6-34` (table `business_ideas`) Columns: `id` PK; `user_id` FK→ `users.id` NOT NULL; `conversation_id` FK→ `conversations.id` nullable; `idea_name` String(255) NOT NULL; `idea_summary` Text NOT NULL; `full_response` Text NOT NULL (JSON string of full KIP response); `location` String(255) nullable; `capital_amount` Float nullable; `skills` Text nullable; `generated_by_kip` Boolean default True; `accepted` Boolean nullable (tri-state: True/False/None=pending); `decline_reason` Text nullable; `created_at` DateTime default utcnow. `relationship("User", back_populates="business_ideas")` — the only file besides `conversation.py` that declares a bidirectional ORM relationship back to `User`. Represents one candidate business idea suggested to a user, optionally generated within a specific chat `Conversation`. Feeds into `BusinessLaunchPlan.idea_id` when accepted. **Note:** `conversation_id` FK has no matching `relationship()` to `Conversation` (one-directional FK only), and neither `user_id` nor `conversation_id` is indexed. The `accepted` tri-state Boolean (True/False/None) is a slightly unusual modeling choice — an enum/status string would be more explicit than relying on NULL-as-pending semantics.

conversation.py

What this file models: The general-purpose chat conversation and message history between a user and KIP (distinct from the business-plan-specific chat in `business_chat_model.py` and the startup-specific chat in `startup_chat.py`).

- **Conversation** — `conversation.py:6-16` (table `conversations`) Columns: `id` PK; `user_id` FK→ `users.id` NOT NULL; `title` String(255) default "New Conversation"; `created_at` / `updated_at` DateTime (onupdate=utcnow). Relationships: `user = relationship("User", back_populates="conversations")`; `messages = relationship("Message", back_populates="conversation", cascade="all, delete-orphan")`. Represents a chat thread owned by a user; deleting a conversation cascades to delete its messages.
- **Message** — `conversation.py:18-27` (table `messages`) Columns: `id` PK; `conversation_id` FK→ `conversations.id` NOT NULL; `role` String(20) NOT NULL ("user" / "assistant"); `content` Text NOT NULL; `created_at` DateTime default utcnow. Relationship: `conversation = relationship("Conversation", back_populates="messages")`. Represents a single chat turn within a `Conversation`. **Note:** Neither `user_id` on `Conversation` nor `conversation_id` on `Message` is indexed, despite being the primary access patterns (list a user's conversations; list a conversation's messages).

enhanced_logs.py

What this file models: Sprint 6's expanded, five-category daily business logging system (financial, customers/sales, operations/inventory, marketing/leads, time/productivity) plus ML-derived predictions, weekly rollups, and a market-context survey — the analytics/BI layer built on top of `BusinessLaunchPlan`.

- **BusinessLogTemplate** — `enhanced_logs.py:10-21` (table `business_log_templates`) Columns: `id` PK; `plan_id` FK→ `business_launch_plans.id` NOT NULL, **unique** (one template per plan); `business_type` String(100) nullable; `custom_fields` Text NOT NULL (JSON); `created_at` DateTime default utcnow. Represents a per-business customization of which extra sub-fields appear in that business's daily log form (semi-adaptive log UI).
- **EnhancedDailyLog** — `enhanced_logs.py:24-85` (table `enhanced_daily_logs`) Columns: `id` PK; `plan_id` FK→ `business_launch_plans.id` NOT NULL; `user_id` FK→ `users.id` NOT NULL; `log_date` DateTime default utcnow; `day_number` Integer nullable; ~30 metric columns across 5 categories (revenue, cash/mobile money, expenses, customer counts, walk-ins/conversions, hours/stock/downtime, marketing spend/leads, time breakdown, daily win/lesson), plus `coaching_response` / `coaching_at`. Represents the full daily operating record for a business — the richer replacement for `DailyBusinessLog` (`business_dashboard.py`). Feeds `WeeklyReview` and `MLPrediction`. **Note:** Coexistence of `DailyBusinessLog` and `EnhancedDailyLog` (both FK to `business_launch_plans`, both storing daily revenue/expenses/customers) is a clear duplication risk — comments confirm this ("Replaces/extends the basic `DailyBusinessLog`") but the old table/model was not removed. No indexes on `plan_id` / `user_id`.
- **WeeklyReview** — `enhanced_logs.py:88-122` (table `weekly_reviews`) Columns: `id` PK; `plan_id` / `user_id` FKs NOT NULL; `week_start` / `week_end` DateTime NOT NULL; aggregated KPIs (`total_revenue`, `total_expenses`, `net_profit`, `avg_daily_customers`, `avg_sale_value`, `total_logs_filed`); week-over-week `revenue_change_pct` / `customer_change_pct`; ML outputs (`top_selling_product`, `biggest_expense_cat`, `peak_day`, `trend_status` default "neutral"); `kip_summary` Text; `predictions_json` Text; `created_at`. Represents an auto-generated weekly rollup/summary of a business's performance, computed from `EnhancedDailyLog` rows.

- **MLPrediction** — `enhanced_logs.py:125-159` (table `ml_predictions`) Columns: `id` PK; `plan_id` / `user_id` FKs NOT NULL; `generated_at` DateTime default utcnow; `revenue_forecast_7d` Text (JSON array); `forecast_confidence` Float; `customer_trend` / `customer_trend_pct`; `peak_day_of_week` / `peak_hour_range`; `days_until_stockout` / `low_stock_items`; `expense_anomaly` Boolean default False + `anomaly_category` / `anomaly_detail`; `breakeven_progress_pct` / `days_to_breakeven`. Represents a rolling snapshot of ML-derived forecasts/insights for a business, regenerated on each new log submission.
- **MarketSurvey** — `enhanced_logs.py:162-213` (table `market_surveys`) Columns: `id` PK; `user_id` FK NOT NULL; `plan_id` FK→ `business_launch_plans.id` nullable; `location` String(255) nullable; `completed_at` DateTime nullable; ~25 survey fields across location/infrastructure, customer profile, competition, supply chain, and business environment sections; `additional_notes` Text; `created_at` / `updated_at`. Represents a one-time (or periodic) market-context questionnaire a user fills in about their business's physical location, used to enrich KIP's contextual advice. **Note:** None of the five classes in this file declare a single `relationship()` — everything is raw FK columns, and none of the FK columns (`plan_id`, `user_id`) are indexed anywhere in this file despite being the dominant query pattern ("get all logs/reviews/predictions for plan X"). `plan_id` is inconsistently nullable here (nullable in `MarketSurvey`, NOT NULL elsewhere in the same file).

enterprise_models.py

What this file models: Sprint 13's multi-branch/enterprise account layer, letting one user act as a central manager owning multiple business branches, each potentially run by a different branch manager, plus an in-app notification system for branch managers.

- **EnterpriseBusiness** — `enterprise_models.py:13-30` (table `enterprise_businesses`) Columns: `id` PK; `owner_user_id` FK→ `users.id` NOT NULL, indexed; `name` String(200) NOT NULL; `description` Text nullable; `industry` String(100) nullable; `headquarters` String(200) nullable; `branch_count` Integer default 0; `is_active` Boolean default True; `created_at` / `updated_at`. Represents the top-level corporate entity owned by one user, containing multiple branches. **Note:** `branch_count` is a denormalized counter column with no visible trigger/service-layer enforcement in this file — risk of drifting out of sync with actual `EnterpriseBranch` row count.
- **EnterpriseBranch** — `enterprise_models.py:33-52` (table `enterprise_branches`) Columns: `id` PK; `enterprise_id` FK→ `enterprise_businesses.id` NOT NULL, indexed; `plan_id` FK→ `business_launch_plans.id` nullable; `branch_name` String(200) NOT NULL; `location` String(200) nullable; `specialty` String(300) nullable; `branch_manager_user_id` FK→ `users.id` nullable, indexed; `manager_email` String(255) nullable; `monthly_target` Float nullable; `is_active` Boolean default True; `created_at`. Represents a single branch/location of an enterprise, optionally linked to its own `BusinessLaunchPlan` dashboard and optionally assigned a manager `User`. **Note:** `manager_email` is stored redundantly alongside `branch_manager_user_id` FK ("stored for reference" per comment) — a denormalization that can go stale if the manager's email changes in `users.email`.
- **BranchNotification** — `enterprise_models.py:54-68` (table `branch_notifications`) Columns: `id` PK; `user_id` FK→ `users.id` NOT NULL, indexed; `enterprise_id` FK→ `enterprise_businesses.id` NOT NULL (not indexed); `branch_id` FK→ `enterprise_branches.id` NOT NULL (not indexed); `message` Text NOT NULL; `is_read` Boolean default False; `created_at`. Represents an in-app banner notification shown to a branch manager on login. **Note:** `enterprise_id` and `branch_id` lack indexes despite `user_id` being indexed in the same table — inconsistent indexing within a single class. No `relationship()` declarations anywhere in the file; all navigation is FK-only.

models_briefing.py

What this file models: A single-table feature storing KIP-generated "Economic Briefing" news/analysis cards about Zambia's macroeconomic environment (fiscal policy, etc.), unrelated to any specific user — a content/editorial table rather than a user-transactional one.

- **EconomicBriefing** — `models_briefing.py:40-83` (table `economic_briefings`) Columns: `id` PK autoincrement; `title` String(200) NOT NULL; `source` String(200) NOT NULL; `source_url` String(500) nullable; `category` String(50) NOT NULL default "fiscal_policy"; `importance` String(10) NOT NULL default "MEDIUM"; `emoji` String(10) nullable default "🇿🇲"; `headline_plain` String(500) NOT NULL; `explanation` Text NOT NULL; `impact_ordinary` / `impact_business` / `kip_take` Text nullable; `image_search_term` String(100) nullable default "Zambia economy"; `date_published` String(20) nullable; `generated_date` String(20) NOT NULL (YYYY-MM-DD as string); `generated_at` DateTime NOT NULL default utcnow. `__table_args__` defines `Index("ix_briefings_generated_date", "generated_date")` explicitly (the only file among the 11 to use an explicit `Index` construct rather than `Column(index=True)`). Also defines a `to_dict()` serialization helper method. No FKs/relationships — this is a standalone content table with no `user_id`, so briefings are global/shared across all users rather than personalized. **Note:** `generated_date` is stored as a `String(20)` rather than a proper `Date` / `DateTime` type, which prevents native date range queries/comparisons and relies on lexicographic string ordering (works only because of the enforced YYYY-MM-DD format). The file's docstring is a leftover implementation/migration note ("ADD THIS CLASS to your existing app/models.py file... Also add the Alembic migration or run: ALTER

TABLE”) suggesting this table may have been bolted on outside the normal migration workflow — worth confirming Alembic history actually covers it.

news.py

What this file models: External news/market-data ingestion (economy news articles and live financial rates like USD/ZMW, fuel, copper) plus a per-user alert system notifying users when a news item or rate change is relevant to their business.

- **NewsArticle** — `news.py:5-15` (table `news_articles`) Columns: `id` PK; `headline` String(500) NOT NULL; `summary` Text nullable; `url` String(1000) unique, NOT NULL; `source_name` String(100) NOT NULL; `category` String(60) default `"general_economy"`; `published_at` DateTime nullable; `fetched_at` DateTime default utcnow; `is_active` Boolean default True. Represents one scraped/ingested news article relevant to Zambian economy/business.
- **LiveRate** — `news.py:17-25` (table `live_rates`) Columns: `id` PK; `rate_type` String(50) NOT NULL (`usd_zmw` / `petrol` / `diesel` / `copper`); `value` Float NOT NULL; `unit` String(30) default `" "`; `direction` String(10) default `"stable"`; `previous` Float nullable; `fetched_at` DateTime default utcnow. Represents a snapshot reading of a tracked financial/commodity rate.
Note: No unique constraint or index on `rate_type`, so historical rows for the same `rate_type` accumulate with no way to efficiently fetch “latest per type” without a full scan/ordering by `fetched_at`.
- **KipAlert** — `news.py:27-38` (table `kip_alerts`) Columns: `id` PK; `user_id` FK→ `users.id` NOT NULL; `article_id` FK→ `news_articles.id` nullable; `rate_type` String(50) nullable; `headline` String(500) NOT NULL; `kip_message` Text NOT NULL; `business_name` String(255) nullable; `category` String(60) nullable; `is_read` Boolean default False; `created_at` DateTime default utcnow. Represents a personalized KIP-generated alert for a user, triggered by either a `NewsArticle` or a `LiveRate` change (`rate_type` used when not tied to an article). **Note:** `user_id` and `article_id` are not indexed despite being the two primary FK lookup paths (“get all unread alerts for user X”). No relationships declared in this file.

startup_chat.py

What this file models: A separate, dedicated chat thread type for pre-launch “startup questions” (distinct from both the general `Conversation` in `conversation.py` and the plan-bound chat in `business_chat_model.py`) — structurally almost identical to `Conversation` / `Message` but namespaced separately.

- **StartupConversation** — `startup_chat.py:6-16` (table `startup_conversations`) Columns: `id` PK; `user_id` FK→ `users.id` NOT NULL; `title` String(255) default `"Startup question"`; `step_emoji` String(8) default `"🚀"`; `created_at` / `updated_at`. Relationship: `messages = relationship("StartupMessage", back_populates="conversation", cascade="all, delete-orphan")`. Represents a chat thread specifically about early-stage startup questions, before a business idea/plan exists. **Note:** No `user = relationship(...)` back to `User` (unlike `Conversation`, which does have it), and `User` has no corresponding `startup_conversations` relationship either — inconsistent with how `Conversation` wires its `User` relationship in the same codebase.
- **StartupMessage** — `startup_chat.py:18-27` (table `startup_messages`) Columns: `id` PK; `conversation_id` FK→ `startup_conversations.id` NOT NULL; `role` String(20) NOT NULL; `content` Text NOT NULL; `created_at` DateTime default utcnow. Relationship: `conversation = relationship("StartupConversation", back_populates="messages")`. Represents one message turn in a startup conversation; mirrors `Message` in `conversation.py` almost exactly (near-duplicate table design). **Note:** Neither `user_id` on `StartupConversation` nor `conversation_id` on `StartupMessage` is indexed. This near-duplication of `Conversation` / `Message` (3 tables total across 2 files doing very similar things: `conversations.py`, `startup_chat.py`, plus `business_chat_model.py`'s chat table) is a schema-consolidation opportunity worth flagging.

subscription_models.py

What this file models: Sprint 15’s billing/monetization layer — a user’s subscription tier/status (supporting both Stripe and Flutterwave as payment providers), daily rate-limit usage tracking, and a full payment attempt/outcome ledger.

- **UserSubscription** — `subscription_models.py:10-31` (table `user_subscriptions`) Columns: `id` PK; `user_id` FK→ `users.id` NOT NULL, **unique**, indexed (enforces one subscription row per user); `plan_tier` String(20) default `"free"`; `status` String(20) default `"active"`; `trial_ends_at` / `current_period_start` / `current_period_end` DateTime nullable; `stripe_customer_id` / `stripe_subscription_id` String(100) nullable; `flw_customer_id` / `flw_subscription_code` String(100) nullable; `payment_method` String(20) nullable; `amount_zmw` Float default 0.0; `created_at` / `updated_at`. Represents a user’s current subscription/billing state, dual-provider (Stripe for card, Flutterwave for local/mobile money). **Note:** `plan_tier` here duplicates `User.plan_tier` (`user.py:14`) — two separate “source of truth” columns for the same concept across two tables, with no visible sync mechanism in this file.

- **DailyUsage** — `subscription_models.py:34-44` (table `daily_usage`) Columns: `id` PK; `user_id` FK→ `users.id` NOT NULL, indexed; `usage_date` String(10) NOT NULL (YYYY-MM-DD string, not Date type); `ideas_generated` / `chat_messages` Integer default 0; `created_at` / `updated_at` . Represents a per-user-per-day counter of rate-limited feature usage (idea generation, chat messages) for enforcing free-tier limits. **Note:** No unique/composite constraint on `(user_id, usage_date)` , so nothing at the DB level prevents duplicate rows for the same user+day; also `usage_date` as String rather than Date, same pattern seen in `EconomicBriefing.generated_date` .
- **PaymentRecord** — `subscription_models.py:47-61` (table `payment_records`) Columns: `id` PK; `user_id` FK→ `users.id` NOT NULL, indexed; `provider` String(20) NOT NULL; `provider_ref` String(200) nullable; `amount_zmw` Float NOT NULL; `currency` String(5) default "ZMW" ; `status` String(20) default "pending" ; `plan_tier` String(20) default "premium" ; `months` Integer default 1; `description` Text nullable; `created_at` . Represents a single payment attempt/transaction record (success, failure, or refund) tied to a user's subscription purchase. **Note:** No index on `provider_ref` despite it being the natural lookup key for webhook reconciliation (matching an incoming Stripe/Flutterwave webhook event back to a `PaymentRecord`).

user.py

What this file models: The central identity/account table for the whole platform — every other user-scoped table's `user_id` FK ultimately points here.

- **User** — `user.py:6-22` (table `users`) Columns: `id` PK; `full_name` String(120) NOT NULL; `email` String(255) unique, indexed, NOT NULL; `hashed_password` String(255) NOT NULL; `is_active` Boolean default True; `plan_tier` String(20) default "free" ; `is_verified` Boolean default False; `verification_token` String(255) unique, indexed, nullable; `verification_token_expires` DateTime nullable; `created_at` / `updated_at` . Relationships: `conversations = relationship("Conversation", back_populates="user")` ; `business_ideas = relationship("BusinessIdea", back_populates="user")` . Represents a registered KIP account holder. Is the root of virtually every other table in the schema via `user_id` FKs (business ideas, plans, logs, chats, subscriptions, alerts, enterprise ownership, etc.), but only two of those relationships (`conversations` , `business_ideas`) are wired as ORM `relationship()` objects here — the rest (business_dashboard, enhanced_logs, enterprise_models, news, subscription_models, business_chat_model, startup_chat) are FK-only and must be queried manually. **Note:** `plan_tier` is duplicated in `UserSubscription.plan_tier` (subscription_models.py:15) with no enforced consistency. Highly incomplete relationship coverage on the central table — `User` has no `relationship()` back to `BusinessLaunchPlan` , `EnhancedDailyLog` , `UserSubscription` , `KipAlert` , `EnterpriseBusiness` , `StartupConversation` , etc., despite all of them FK-referencing `users.id` ; this is inconsistent with the pattern established for `conversations` / `business_ideas` and likely forces routes/services to do manual `.filter(Model.user_id == user.id)` queries throughout the codebase instead of using `user.xxx` .

Entity-Relationship Narrative

`User` (user.py) is the root of the schema: every other table's ownership traces back to `users.id` , though only `conversations` and `business_ideas` are wired as true ORM relationships — the rest are FK-only. A user chats generally via `Conversation` → `Message` (conversation.py), asks pre-launch questions via `StartupConversation` → `StartupMessage` (startup_chat.py), and receives KIP-generated `BusinessIdea` suggestions (business_idea.py), optionally within a `Conversation` . Accepting an idea creates a `BusinessLaunchPlan` (business_dashboard.py) — the central operational hub — which accumulates `DailyBusinessLog` / `CoachingEntry` (legacy) or the richer `EnhancedDailyLog` , `WeeklyReview` , `MLPrediction` , `MarketSurvey` (enhanced_logs.py), plus its own `BusinessChatMessage` / `BusinessPlanVersion` thread (business_chat_model.py). A user can also own an `EnterpriseBusiness` containing multiple `EnterpriseBranch` rows, each optionally linked to its own `BusinessLaunchPlan` and manager `User` , with `BranchNotification` alerting managers (enterprise_models.py). Separately, `NewsArticle` / `LiveRate` (news.py) feed personalized `KipAlert` rows per user, while `UserSubscription` , `DailyUsage` , and `PaymentRecord` (subscription_models.py) track billing state, each independently FK'd to `users.id` . `EconomicBriefing` (models_briefing.py) is the only fully standalone, non-user-scoped table.

Static Reference Data (backend/app/data/)

backend/app/data/initial_knowledge.py

Purpose: Seed content for KIP's RAG knowledge base. It is the "undergraduate curriculum" of economics/finance/business texts that gets embedded into ChromaDB the first time the knowledge base is initialized (empty collection).

Shape: A single top-level list `INITIAL_KNOWLEDGE_TEXTS` (lines 11–393) of dicts, one per knowledge document, each with keys: `id` (str, e.g. `"econ_basics_01"`), `category` (str: `"economics"`, `"finance"`, `"business"`, or `"zambia"`), `subcategory` (str), `title` (str), `content` (long triple-quoted prose string, Zambia-contextualized explanations with worked examples), `zambia_relevance` (str: `"high"` / `"very_high"` / etc.)

Entry count: 9 documents.

Consumption: Not imported anywhere under `backend/app` at runtime. It is imported by the standalone seeding/ingestion script `build_kip_sprint2.py:1112` (`from app.data.initial_knowledge import INITIAL_KNOWLEDGE_TEXTS`), which chunks and embeds each `doc["content"]` into ChromaDB on first setup (`build_kip_sprint2.py:1165-1177` , `_seed_initial_knowledge`). At request time, the backend instead reads the already-persisted ChromaDB collection via `backend/app/services/knowledge_base.py` (`get_knowledge_base().search(...)` , used in `backend/app/services/kip_engine.py:147-148`), so this module is purely a one-time data-seeding source, not a runtime dependency.

backend/app/data/town_coordinates.py

Purpose: Geocoding/location registry for Zambian towns and neighbourhoods, used to query OpenStreetMap (Overpass API) for local business density data, plus a taxonomy of OSM tags KIP cares about for competitive/market analysis.

Shape: - `TOWN_COORDS` (dict, lines 11–127): keys are lowercase town/neighbourhood name strings (e.g. `"lusaka"`, `"kitwe cbd"`, `"matero"`), values are dicts with `lat` (float), `lon` (float), `radius_m` (int search radius in metres), `province` (str). Organized by province in comment-delimited blocks (Lusaka, Copperbelt, Southern, Eastern, Central, North-Western, Northern, Luapula, Western, Muchinga). - `OSM_CATEGORIES` (dict, lines 131–179): keys are category slugs (`"food_and_drink"`, `"retail_shops"`, `"services"`, `"financial"`, `"health"`, `"education"`, `"transport"`, `"markets"`); values are dicts with `tags` (list of OSM `amenity=` / `shop=` tag strings), `kip_label` (human-readable str), `competition_signal` (bool). - `resolve_location(location_string)` (function, lines 181–199): matches a free-text location to the best `TOWN_COORDS` key (exact match, else longest partial substring match).

Entry count: 93 entries in `TOWN_COORDS` ; 8 categories in `OSM_CATEGORIES` .

Consumption: Imported in `backend/app/services/map_service.py:20` — `from app.data.town_coordinates import TOWN_COORDS, OSM_CATEGORIES, resolve_location` . `resolve_location` is called at `map_service.py:212` inside `get_map_context()` to turn a user-supplied location string into a cache key/coordinate pair; `OSM_CATEGORIES` is used at `map_service.py:163-173` to label and filter business counts fetched from OSM/cache. Also imported locally inside `backend/app/services/kip_engine.py:70` (`from app.data.town_coordinates import TOWN_COORDS`).

backend/app/data/town_profiles.py

Purpose: Curated "ground-truth" local intelligence per Zambian town/neighbourhood (economic character, income levels, market gaps, competition density, financing sources) used to enrich KIP's contextual reasoning/system-prompt injection. Many entries are explicitly flagged as placeholders needing founder review.

Shape: - `TOWN_PROFILES` (dict, lines 18–300): keys are lowercase city slugs (`"kitwe"`, `"ndola"`, `"chingola"`, `"lusaka"`, `"livingstone"`, `"chipata"`, `"kabwe"`, `"solwezi"`). Each value is a dict with keys such as `city`, `province`, `classification`, `population_estimate`, `economic_character`, `income_distribution`, `major_employers` (list), `key_markets` (list), `financing_sources` (list), `needs_founder_review` (bool), and optionally `neighbourhoods` (nested dict, present for the larger cities: Kitwe, Ndola, Lusaka). Each neighbourhood entry is itself a dict keyed by neighbourhood slug (e.g. `"riverside"`, `"nkana-east"`, `"kitwe cbd"`, `"parklands"`), with fields like `classification`, `income_bracket`, `population_density`, `dominant_occupations` (list), `avg_household_income_zmw` (int/None), `transport_links`, `commercial_rental_zmw_month` (dict), `known_market_gaps` (list), `competition_density` (dict), `customer_profile`, `notes` (often `"NEEDS_FOUNDER_REVIEW"`). - `get_town_profile(location_string)` (function, lines 303–381): fuzzy-matches a location string against `TOWN_PROFILES` city keys/names and, if matched, nested neighbourhood keys, then formats the matched data into a human-readable multi-line string for prompt injection.

Entry count: 8 top-level city profiles; 3 of them (kitwe, ndola, lusaka) carry nested `neighbourhoods` sub-dicts (4, 2, and additional entries respectively).

Consumption: Imported in `backend/app/services/kip_engine.py:14` — `from app.data.town_profiles import get_town_profile` — and called at `kip_engine.py:154` (`town_profile = get_town_profile(factors["location"])`) to build the `combined_location` context string

that's merged with map intelligence and injected into the conversation before calling the LLM.

4. Backend Deep Dive — Auth, Chat & Core Routes

This chapter covers `backend/app/routes/auth.py`, `auth-stripe.py`, `chat.py`, `dashboard.py`, `ideas.py`, `language_support.py`, `i18n_routes.py`, `startup_chat.py`, `add_own_business.py`, and `briefings.py`.

auth.py

What this router is for: This is the router actually mounted at `/api/auth` in `backend/main.py` and is the live authentication system for KIP. It handles account registration (with email verification), login, fetching the current user's profile, and email verification/resend flows. It issues JWT bearer tokens via `create_access_token` and relies on `app/security.py` for password hashing/verification and the `get_current_user` dependency.

POST /register

`auth.py:25-47`

```
@router.post("/register", response_model=Token, status_code=status.HTTP_201_CREATED)
def register(payload: UserRegister, background_tasks: BackgroundTasks, db: Session = Depends(get_db)):
```

Creates a new `User` from `UserRegister` (full name, email, password). Rejects duplicate emails with 400. Hashes the password with `hash_password`, normalizes email to lowercase, and generates a 24-hour email-verification token via the local helper `_issue_verification_token` (`auth.py:18-22`, uses `secrets.token_urlsafe(32)`). Sends the verification email asynchronously through `BackgroundTasks` (`send_verification_email`), so the HTTP response isn't blocked on SMTP. Returns a `Token` (JWT access token + `UserOut`) immediately — meaning the user is logged in even before verifying their email. No auth dependency (this is the entry point).

Note: No password strength/length validation is visible in this file — it depends entirely on whatever constraints `UserRegister` (in `app/schemas`) enforces; not verified here.

POST /login

`auth.py:49-59`

```
@router.post("/login", response_model=Token)
def login(payload: UserLogin, db: Session = Depends(get_db)):
```

Looks up the user by lowercased email and verifies the password with `verify_password`. Returns a generic 401 ("Incorrect email or password") for both "user not found" and "wrong password" — good practice, avoids leaking which emails are registered. On success, issues a fresh JWT and returns `Token`.

GET /me

`auth.py:61-64`

```
@router.get("/me", response_model=UserOut)
def get_me(current_user: User = Depends(get_current_user)):
```

Simplest endpoint in the file: requires a valid bearer token (`get_current_user`) and just echoes back the authenticated user's profile as `UserOut`. No DB query beyond what `get_current_user` already did.

GET /verify-email

`auth.py:66-79`

```
@router.get("/verify-email")
def verify_email(token: str, db: Session = Depends(get_db)):
```

Public endpoint (no auth) — the `token` query parameter comes from the link in the verification email. Looks up the user by `verification_token`, checks it hasn't expired, then sets `is_verified = True` and clears the token fields. Returns a simple success message.

Note: Token lookup uses an exact-match DB filter on `verification_token` ; because tokens are generated with `secrets.token_urlsafe(32)` this is cryptographically strong, but there's no rate-limiting visible on this public endpoint, so it's technically brute-forceable (though infeasible given token entropy).

POST /resend-verification

`auth.py:81-95`

```
@router.post("/resend-verification")
def resend_verification(
    background_tasks: BackgroundTasks,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
```

Requires login. Blocks with 400 if the user is already verified. Otherwise generates a new token (reusing `_issue_verification_token`) and re-sends the email in the background.

auth-stripe.py

What this router is for: According to its own docstring, this is a “Sprint 15 update” to auth that activates a 30-day premium trial on registration and keeps a user's `plan_tier` in sync with a subscription service on login/ `/me` . **This file is not wired into the app** — `backend/main.py` only imports and mounts `auth.router` (from `auth.py`) at `/api/auth` ; there is no reference to `auth-stripe` or `auth_stripe` anywhere in `main.py` . It appears to be an in-progress/parked feature branch file sitting in the routes folder, not live code. It also duplicates `/register` , `/login` , and `/me` , which would collide with `auth.py` 's routes if it were ever mounted on the same prefix.

POST /register

`auth-stripe.py:20-58`

```
@router.post("/register", response_model=Token)
def register(req: UserRegister, db: Session = Depends(get_db)):
```

Creates the user directly with `plan_tier="premium"` and `is_active=True` (no email verification flow at all — a regression compared to `auth.py`). After committing the user, it does a local import of `app.services.subscription_service` and calls `ensure_subscription_tables(db)` then `activate_trial(user, db)` inside a bare `try/except Exception` that only prints to stdout on failure (`auth-stripe.py:41-46`) — trial activation failures are silently swallowed from the caller's perspective. Issues a token via `create_access_token({"sub": str(user.id)})` and returns a hand-built dict (not going through `UserOut`).

Note (critical bug): `app/security.py` 's `create_access_token(user_id: int)` builds the JWT payload as `{"sub": str(user_id), "exp": ...}` . This route calls it with a **dict** — `create_access_token({"sub": str(user.id)})` — so `user_id` inside that function is the dict itself, and `str(user_id)` becomes the Python string representation of the dict (e.g. `"{'sub': '5'}"`), not the numeric user ID. `get_current_user` then tries `int(payload.get("sub"))` , which raises `ValueError` and is caught, always returning 401 “Invalid or expired session.” **Every token minted by this file's `/register` and `/login` would be permanently unusable for authentication.** This is on top of it being unmounted; it's a second reason this file cannot work as-is.

Note: Bare `except Exception` blocks around subscription-service calls (here and in `/login` , `/me`) hide all errors including programming bugs, not just expected “table doesn't exist yet” cases.

POST /login

`auth-stripe.py:61-98`

```
@router.post("/login", response_model=Token)
def login(req: UserLogin, db: Session = Depends(get_db)):
```

Verifies credentials like `auth.py` , but additionally checks `user.is_active` and returns 403 if false. Then tries to sync `user.plan_tier` from `get_effective_tier(user, db)` (subscription service), committing if it changed. Same broken `create_access_token({"sub": ...})` call as `/register` .

GET /me

auth-stripe.py:101-125

```
@router.get("/me", response_model=UserOut)
def get_me(current_user: User = Depends(get_current_user), db: Session = Depends(get_db)):
```

Requires auth. Tries to fetch `effective_tier` and `trial_info` from the subscription service and returns an enriched dict including `trial_info`; falls back to a plain dict with `trial_info: {"on_trial": False}` if the subscription service throws.

Note: Because this router is never mounted, none of the above executes in production; it should be treated as reference/WIP code, not documentation of live behavior.

chat.py

What this router is for: Powers the main K-BIG-1 conversational chat feature (mounted at `/api/chat`). It manages

`Conversation` / `Message` records per user, forwards user messages plus recent history to the AI engine (`generate_kip_response` in `app/services/kip_engine.py`), and persists the assistant's reply. All endpoints require a logged-in user.

POST /send

chat.py:17-72

```
@router.post("/send", response_model=ChatResponse)
async def send_message(
    payload: ChatMessage,
    request: Request,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):
```

Core chat endpoint. Reads the `X-KIP-Language` header via `get_language_from_request` (from `language_support.py`) to decide English vs Bemba. If `payload.conversation_id` is given, loads that conversation scoped to `current_user.id` (404 if not found/not owned — good tenant isolation). Otherwise creates a new `Conversation`, using the first 60 characters of the message as its title. Saves the user's message as a `Message` row, then loads the most recent 12 messages (`order_by(Message.id.desc()).limit(12)`), then reversed into chronological order) as conversation history. Calls `await generate_kip_response(user_message=..., history=history[:-1], user=current_user, db=db, lang=lang)` — note `history[:-1]` drops the message just saved (since it's passed separately as `user_message`) — receiving back `(reply, idea_detected)`. Saves the assistant reply as another `Message`, bumps `conv.updated_at` so it sorts to the top of conversation lists, and returns `ChatResponse` with `conversation_id`, `reply`, and `idea_saved`.

Note: The `history` query limits to the 12 most recent messages per call — for very long conversations this is a bounded, reasonable choice, but it's not paginated/configurable and always re-fetches from the DB on every message (no caching).

GET /conversations

chat.py:75-83

```
@router.get("/conversations", response_model=List[ConversationOut])
def get_conversations(
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):
```

Returns all of the current user's conversations, most recently updated first. No pagination — for a heavy user this returns unbounded rows.

GET /conversations/{conv_id}

chat.py:86-98

```
@router.get("/conversations/{conv_id}", response_model=ConversationOut)
def get_conversation(
    conv_id: int,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):
```

Fetches a single conversation, scoped to the owning user (404 otherwise). Note: unlike `startup_chat.py`, `chat.py` has no `DELETE /conversations/{conv_id}` endpoint.

dashboard.py

What this router is for: Small router (mounted at `/api/dashboard`) that aggregates data for the user's home dashboard: summary stats, a preview of recent business ideas, and a computed "KIP score." Every endpoint requires authentication and reads user-scoped data only.

GET /stats

`dashboard.py:13-20`

```
@router.get("/stats", response_model=DashboardStats)
def get_dashboard_stats(current_user: User = Depends(get_current_user), db: Session = Depends(get_db)):
```

Runs three separate `COUNT` queries against `Conversation` / `BusinessIdea` scoped to `current_user.id`, plus `current_user.created_at`, and packages them into `DashboardStats`.

Note: Three sequential DB round-trips where a single aggregate query (or one query with conditional aggregates) could compute all counts at once — not a correctness issue, just an efficiency one for a hot dashboard endpoint.

GET /recent-ideas

`dashboard.py:22-31`

```
@router.get("/recent-ideas")
def get_recent_ideas(current_user: User = Depends(get_current_user), db: Session = Depends(get_db)):
```

Fetches the user's 3 most recent `BusinessIdea` rows, then **for each idea individually queries** `BusinessLaunchPlan` to find a matching plan (`for i in ideas: db.query(BusinessLaunchPlan).filter(...).first()`). Builds a plain list of dicts with idea id, name, location, capital amount, accepted flag, and `plan_id` (or `None`).

Note (N+1 query): This is a textbook N+1 pattern — 1 query for ideas + up to 3 more queries (one per idea) for plans. Bounded by the `.limit(3)` so the impact is small here, but it's a pattern worth flagging since it would scale badly if the limit were raised. Also has no `response_model`, so the response shape is only enforced by convention, not validated/documentated via Pydantic.

GET /score

`dashboard.py:33-35`

```
@router.get("/score")
def get_kip_score(current_user: User = Depends(get_current_user), db: Session = Depends(get_db)):
```

Delegates entirely to `calculate_kip_score(current_user.id, db)` from `app/services/kip_score.py` and returns whatever it produces. No `response_model`, so the exact schema of the "KIP score" isn't visible from this file alone.

ideas.py

What this router is for: Mounted at `/api/ideas`, this router lets a logged-in user list the business ideas KIP has generated for them and record feedback (accept/decline, with an optional decline reason) on a specific idea.

GET /

ideas.py:13-22

```
@router.get("/", response_model=List[IdeaOut])
def get_my_ideas(
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):
```

Returns all `BusinessIdea` rows for the current user, newest first. No pagination — could return an unbounded list for prolific users.

POST /{idea_id}/feedback

ideas.py:24-45

```
@router.post("/{idea_id}/feedback")
def submit_feedback(
    idea_id: int,
    feedback: IdeaFeedback,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):
```

Looks up the idea scoped to `idea_id` and `user_id == current_user.id` (404 if not found/not owned — correct tenant isolation, prevents one user from tampering with another's idea via ID guessing). Sets `idea.accepted = feedback.accepted`; if declined and a `decline_reason` was supplied, stores it for KIP's continuous learning. Returns a simple status dict, not a full `IdeaOut`.

Note: No `response_model` on this endpoint, so callers get an untyped `{"status": ..., "idea_id": ...}` — inconsistent with the `GET /` endpoint's typed response.

language_support.py

What this router is for: Not actually a router (no `APIRouter()` instantiated, no `@router` decorators) — despite living in the `routes/` folder, this is a shared **helper module** used by other routers (`chat.py`, `startup_chat.py`, and presumably BizSim/KIP Learn) to support the English/Bemba bilingual feature. It reads a custom `X-KIP-Language` header and produces prompt-injection text to steer Claude's responses into Bemba-English code-switching.

get_language_from_request(request: Request) -> str

language_support.py:20-26

```
def get_language_from_request(request: Request) -> str:
```

Reads the `X-KIP-Language` header, lowercases/strips it, and returns it only if it's in `SUPPORTED_LANGUAGES = {"en", "bem"}` (`language_support.py:17`); otherwise defaults to `"en"`. This is a plain function, not an endpoint — no auth, no DB access, just header parsing.

language_instruction(lang: str) -> str

`language_support.py:29-48` Returns an empty string for anything other than `"bem"`, preserving existing English-only behavior unchanged. For Bemba, returns a detailed instruction block to append to a Claude system prompt, explicitly telling the model to mix Bemba with English business/financial terms (natural Zambian code-switching), keep numbers/currency in K (Kwacha) unchanged, and not do literal word-for-word translation.

get_language_label(lang: str) -> str

`language_support.py:51-53` Trivial lookup dict (`{"en": "English", "bem": "Bemba"}`) with `"English"` as the fallback for unknown codes.

Note: `SUPPORTED_LANGUAGES` here (`{"en", "bem"}`) is a *different, smaller* set than `SUPPORTED_LANGUAGES` imported from `app.services.zedtranslate_service` in `i18n_routes.py` (which supports more languages like `ny`, `toi`, `loz`, `tum`). Two same-named constants with different membership in different modules is a maintenance hazard — easy to import the wrong one.

i18n_routes.py

What this router is for: Mounted at `/api/i18n`, this router serves the full internationalization system for the UI: listing supported languages, serving pre-built/cached UI string bundles per language, on-demand translation of arbitrary or AI-generated text (via a “ZedTranslate” service), and reading/writing the logged-in user’s language preference. It mixes file-based (pre-built JSON) and SQLite-cache-based translation strategies with graceful fallbacks to English.

GET /languages

i18n_routes.py:43-51

```
@router.get("/languages")
def get_languages():
```

Public, no auth, no DB. Returns a hardcoded list of 3 languages (English, Bemba, Nyanja) with code/name/native-name/flag emoji. Purely static data — inconsistent with the larger `SUPPORTED_LANGUAGES` set used elsewhere in the same file.

GET /strings/{lang}

i18n_routes.py:56-103

```
@router.get("/strings/{lang}")
async def get_strings(
    lang: str,
    db: Session = Depends(get_db),
):
```

Public endpoint (no auth) that serves the full UI string bundle for a language, with a multi-tier fallback chain: (1) for `lang == "en"`, serve `data/i18n/en.json`, falling back to `data/kip_strings.json` if that’s missing; (2) for other languages, first validate `lang` is in `SUPPORTED_LANGUAGES` from `zedtranslate_service` (400 if not); (3) try the pre-built `data/i18n/{lang}.json` file (fastest path, no DB); (4) fall back to `get_ui_strings(lang, db)` — a SQLite translation cache; (5) if nothing is cached, return English strings with a `warning` field noting the translations aren’t available; (6) if even English is missing, raise 503.

Note: File reads (`open(...)`) are synchronous/blocking I/O inside an `async def` endpoint — under load this blocks the event loop instead of yielding to other requests. Since these are small local JSON files it’s a minor concern, but it’s a pattern worth knowing about.

POST /translate

i18n_routes.py:108-124

```
@router.post("/translate")
async def translate_text(
    req: TranslateRequest,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
```

Requires auth. Takes `TranslateRequest` (`text`, `target`, `source` defaulting to `"en"`). Short-circuits and returns the input unchanged if `text` is empty/whitespace. Otherwise calls `await translate(req.text, target=req.target, source=req.source, db=db)` from `zedtranslate_service` and returns `{"translated": ..., "lang": ...}`.

Note: No validation that `req.target` is actually in `SUPPORTED_LANGUAGES` before calling `translate()` — unlike `/strings/{lang}` and `/language`, this endpoint trusts the caller’s `target` code entirely.

POST /translate-ai

i18n_routes.py:127-151

```
@router.post("/translate-ai")
async def translate_ai_content(
    req: TranslateRequest,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
```

Same shape as `/translate` but semantically for Claude-generated content (coaching text, ideas, explanations). If `target == "en"`, returns the text unchanged with `ai_translated: False`. Otherwise translates and returns an extra `attribution: "ZedTranslate"` field alongside `ai_translated: True`, presumably so the frontend can show a “translated by ZedTranslate” disclaimer on AI content.

PUT /language

`i18n_routes.py:156-178`

```
@router.put("/language")
def set_language(
    req: LanguageRequest,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
```

Requires auth. Validates `req.language` against a hardcoded list `["en", "bem", "ny", "toi", "loz", "tum"]` (400 if invalid — this list is defined inline here and is *not* the same object as `SUPPORTED_LANGUAGES` imported at the top, another duplicated-source-of-truth issue). Runs a **runtime schema migration**: checks `PRAGMA table_info(users)` for a `language` column and, if missing, executes `ALTER TABLE users ADD COLUMN language TEXT DEFAULT 'en'` on the fly. Then updates the user’s `language` column via raw SQL (`UPDATE users SET language=:lang WHERE id=:uid`) with parameter binding (safe from SQL injection).

Note: Running `ALTER TABLE` as part of a request handler (rather than a proper migration step at startup) is unusual and risky — under concurrent requests from different users hitting this endpoint simultaneously before the column exists, multiple processes could race on the `PRAGMA / ALTER TABLE` check-then-act, though SQLite’s locking would likely serialize this in practice.

GET /language

`i18n_routes.py:181-195`

```
@router.get("/language")
def get_language(
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
):
```

Requires auth. Same `PRAGMA table_info` check as `PUT /language`; if the `language` column doesn’t exist yet, returns `{"language": "en"}` without creating the column. Otherwise raw-SQL selects the user’s stored language, defaulting to `"en"` if null.

Note: Both `PUT` and `GET /language` re-run `PRAGMA table_info(users)` on every single call instead of caching the result or handling this via a real migration — unnecessary per-request overhead.

startup_chat.py

What this router is for: Mounted at `/api/startup-chat`, this is a near-duplicate of `chat.py` but for a separate conversation stream dedicated to guiding a founder through the startup/registration process (as opposed to general chat). It uses its own models (`StartupConversation`, `StartupMessage`) and schemas, but shares the same AI engine (`generate_kip_response`) and language-detection helper. Unlike `chat.py`, it also supports deleting a conversation.

POST /send

`startup_chat.py:17-70`

```

@router.post("/send", response_model=StartupChatResponse)
async def send_startup_message(
    payload: StartupChatMessage,
    request: Request,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):

```

Structurally identical to `chat.py`'s `/send`: resolves/creates a `StartupConversation` scoped to the user, saves the user message, loads the last 12 messages as history, and calls `generate_kip_response`. The distinctive piece is `step_context` handling (`startup_chat.py:52-54`): if `payload.step_context` is set, the message actually sent to the AI engine is rewritten as `f"{payload.step_context}\n\nFounder's question: {payload.message}"` — this lets the frontend inject the current registration-step's context ahead of the literal question. Note this endpoint discards the `idea_detected` second return value from `generate_kip_response` (`reply, _ = await generate_kip_response(...)`), unlike `chat.py` which surfaces it as `idea_saved`.

Note: The stored `Message` / `StartupMessage` content only ever contains the raw `payload.message`, not the `step_context`-augmented `engine_message` — so history reloaded in later turns won't include the step context that was injected for the AI on this turn.

GET /conversations

`startup_chat.py:73-81`

```

@router.get("/conversations", response_model=List[StartupConversationOut])
def get_startup_conversations(
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):

```

Same pattern as `chat.py`: all of the user's startup conversations, most recently updated first, unpaginated.

GET /conversations/{conv_id}

`startup_chat.py:84-96`

```

@router.get("/conversations/{conv_id}", response_model=StartupConversationOut)
def get_startup_conversation(
    conv_id: int,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):

```

Fetches one conversation scoped to the user; 404 if not found/owned.

DELETE /conversations/{conv_id}

`startup_chat.py:99-113`

```

@router.delete("/conversations/{conv_id}")
def delete_startup_conversation(
    conv_id: int,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):

```

Looks up the conversation scoped to the current user (404 if not found/owned), then `db.delete(conv)` and commits. Returns `{"status": "deleted"}`.

Note: Relies on the DB relationship/cascade configuration to clean up associated `StartupMessage` rows when the conversation is deleted; if cascade delete isn't configured on the `StartupConversation` ↔ `StartupMessage` relationship, this could leave orphaned message rows.

add_own_business.py

What this router is for: Mounted at `/api/business` alongside `bd_routes.router`, this router lets a user who already runs a business add it directly to KIP (rather than going through KIP's idea-generation flow). It calls Claude to generate a growth-focused analysis/improvement plan (explicitly *not* a startup plan), then immediately persists both a `BusinessIdea` and a `BusinessLaunchPlan` — with no accept/decline review step, unlike the standard idea flow in `ideas.py`.

POST /add-own

`add_own_business.py:51-122`

```
@router.post("/add-own")
async def add_own_business(
    req: AddOwnBusinessRequest,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):
```

Requires auth. Request body is `AddOwnBusinessRequest` (`add_own_business.py:31-48`): `business_name`, `description` (required); `location`, `main_challenge` (optional strings); `tenure` (defaults to `"just_started"`, mapped through `TENURE_LABELS` into a human-readable phrase like "1–6 months old"); `monthly_revenue` and `capital_invested` (optional floats) with a `field_validator` (`add_own_business.py:40-48`) that coerces incoming strings to floats and silently returns `None` on empty string or conversion failure rather than raising a validation error.

Reads `ANTHROPIC_API_KEY` from the environment and calls the local async helper `_generate_growth_analysis(...)`, which builds a detailed Claude prompt requesting a structured markdown growth plan with fixed sections (Business Assessment, Key Observations, Growth Priorities, Revenue Optimisation, Risks, Compliance Check) in ZMW, calling `client.messages.create(model="claude-sonnet-4-6", max_tokens=1500, ...)` (`add_own_business.py:184-189`). If `api_key` is empty, or the Claude call throws any exception, it falls back to `_fallback_plan(...)` (`add_own_business.py:194-214`) — a static, hardcoded markdown plan with generic advice, so the endpoint never hard-fails even without AI access.

After getting `analysis_text`, it truncates to a 500-character `summary`, creates a `BusinessIdea` with `generated_by_kip=False` (flagging it as user-added, not AI-suggested) and `accepted=True` (pre-accepted, skipping the review step entirely). `db.flush()` is used to get `idea.id` without committing yet. Then it builds a `plan_json` string via `json.dumps({...})` and creates a `BusinessLaunchPlan` row with `status="active"`, `is_profitable=False`. Both rows are committed together.

Note: If the API key is unset, the code degrades gracefully to the fallback plan rather than erroring, which is good defensive design, but there's no user-facing indicator in the response that the plan is a generic fallback vs. an AI-generated one.

Note (no input length/content validation): `business_name` and `description` have no length caps or sanitization before being interpolated directly into the Claude prompt — a very long `description` could inflate token usage/cost, and there's no `max_length` constraint on the Pydantic model.

Note: Both `monthly_revenue` and `capital_invested` can be negative (no `ge=0` constraint), so a user could submit negative revenue/capital and it would flow straight into the DB and the AI prompt uncorrected.

briefings.py

What this router is for: Mounted at `/api/news` (shares the prefix with `news.py`), this router serves KIP's "economic briefings" feature — AI/research-generated daily economic news summaries relevant to Zambian entrepreneurs. It exposes read endpoints for cached briefings and a manual refresh trigger that regenerates them via a background job, guarded by a simple in-process lock. The bottom of the file (`briefings.py:146-187`) documents how to wire up a daily scheduler (APScheduler or Railway Cron) to call `run_daily_briefing` automatically — this scheduler is not implemented, only documented.

GET /briefings

`briefings.py:37-78`

```
@router.get("/briefings")
async def get_briefings(
    date: Optional[str] = Query(None, description="YYYY-MM-DD - defaults to today"),
    db: Session = Depends(get_db),
    _user = Depends(get_current_user),
):
```

Requires auth (the `_user` parameter is unused beyond enforcing login — the endpoint isn't personalized). Accepts an optional `date` query param (defaults to today's UTC date). Queries `EconomicBriefing` rows for that date, ordered by `importance` then `id`. If none exist for the target date, falls back to the single most recent `generated_date` available so the page is "never empty."

POST /briefings/refresh

`briefings.py:81-111`

```
@router.post("/briefings/refresh")
async def refresh_briefings(
    background_tasks: BackgroundTasks,
    db: Session = Depends(get_db),
    _user = Depends(get_current_user),
):
```

Requires auth only — **any logged-in user** can trigger a full briefing regeneration, there's no admin/role check. Uses a module-level global boolean `_refresh_running` (`briefings.py:34`) as a simple in-memory lock: if a refresh is already in progress, immediately returns `{"status": "already_running", ...}`. Otherwise schedules a background task that sets the flag, calls `run_daily_briefing(db)`, and resets the flag in a `finally` block.

Note (missing authorization): This endpoint likely should be restricted to admins/staff, since it triggers an expensive AI research/generation job — as written, any authenticated regular user can call it repeatedly (rate-limited only by the "already running" flag, which only prevents *concurrent* runs, not frequent sequential ones).

Note: `_refresh_running` is a plain Python global/module-level variable, which only works correctly for a single-process deployment. If the backend runs with multiple worker processes, each process has its own copy of this flag, so the "prevent concurrent runs" guarantee breaks down. Also, the `db: Session` dependency captured into the background task may already be closed by the time the background task executes after the response is sent — a common FastAPI pitfall.

GET /briefings/status

`briefings.py:114-143`

```
@router.get("/briefings/status")
async def briefing_status(
    db: Session = Depends(get_db),
    _user = Depends(get_current_user),
):
```

Requires auth. Returns whether a refresh is currently running, how many briefings exist for today, and the timestamp/date of the most recent briefing overall — used for a "Last updated X hours ago" indicator.

Note: Same multi-worker caveat as `/briefings/refresh` — `is_running` reflects only the current process's in-memory state.

5. Backend Deep Dive — Business Operations Routes

This chapter covers `business_chat.py`, `business_dashboard.py`, `capital_access.py`, and `enhanced_logs.py` — the routers behind the “launch and run your business” dashboard.

Router mount points (from `backend/main.py`): `business_chat.router` → `/api/business-chat`; `business_dashboard.router` (imported as `bd_routes`) → `/api/business`; `capital_access.router` → `/api/capital`; `enhanced_logs.router` → `/api/logs`.

`business_chat.py`

What this router is for: This router implements the conversational “chat with your business plan” feature. A logged-in user who has already generated a `BusinessLaunchPlan` can converse with an AI advisor (Claude) that has full context of that specific business (plan details, linked `BusinessIdea`, up to 14 recent `DailyBusinessLog` entries, and an optional `MarketSurvey`). The AI can propose edits to the launch plan itself; the router implements a lightweight “propose → confirm/cancel → apply” state machine using simple keyword matching, persists chat history and plan version history via **raw SQL** against two tables it creates/migrates itself on every request (`business_chat_messages`, `business_plan_versions`), and exposes endpoints to fetch chat history, canned conversation-starter suggestions, and the plan’s version history.

`_ensure_tables(db)` — table bootstrap helper

`business_chat.py:27-79`. Runs `CREATE TABLE IF NOT EXISTS` for `business_chat_messages` and `business_plan_versions`, then does an idempotent “poor-man’s migration” checking `PRAGMA table_info(...)` and running `ALTER TABLE ... ADD COLUMN` if needed. Called at the top of `get_history`, `business_chat`, and `get_plan_versions`. **Note:** Runs on *every request* to those three endpoints. It’s cheap on SQLite but is wasted work per-request rather than a one-time startup migration, and doesn’t handle DB backends other than SQLite (`PRAGMA table_info` is SQLite-specific).

Update-detection heuristics

`business_chat.py:84-106`. `_is_update_suggestion` is unused by the endpoints (dead code — the actual “is this an update?” decision is delegated to Claude via the system prompt). `_is_confirmation` / `_is_rejection` check whether the user’s message starts with/equals a phrase in `CONFIRM_PHRASES` / `REJECT_PHRASES`. **Note:** Fragile heuristic — `"nope".startswith("no")` is `True`, but so is `"nowhere near ready"`, which would be misclassified as a plan-update rejection using simple `startswith` / substring checks.

`_build_context(plan, idea, logs, survey)` — prompt context builder

`business_chat.py:111-149`. Formats plan/idea/daily-log/market-survey data into a plain-text block injected as part of the Claude system prompt. **Note:** All string interpolation of user-controlled data directly into the LLM system prompt with no escaping/sanitization — a classic prompt-injection surface (low real-world severity since it only affects the user’s own session).

`GET /history/{plan_id}`

`business_chat.py:182-205`. Returns the full chat transcript for a plan, ordered oldest-first. Ownership enforced via `WHERE plan_id=pid AND user_id=uid` in parameterized SQL. **Note:** No pagination — for a long-running conversation this returns the entire history in one response every time the chat UI loads.

`POST /chat` — main chat endpoint

`business_chat.py:208-356`. The core endpoint: 1. Ensures tables exist; reads `ANTHROPIC_API_KEY` (503 if missing). 2. Verifies the plan belongs to `current_user` (404 otherwise). 3. Loads context: linked `BusinessIdea`, last 14 `DailyBusinessLog` rows, best-effort `MarketSurvey`. 4. Builds the system prompt, loads last 20 messages of history. 5. Checks for a pending plan-update proposal; a confirmation triggers `_apply_update` (another Claude call rewriting the plan); a rejection returns a canned message. 6. Otherwise calls Claude (`claude-sonnet-4-6`) with the `web_search_20250305` server tool; on exception, retries once without tools; on further exception, raises 500. 7. Detects a `__PLAN_UPDATE_PENDING__: ...` sentinel in the reply via regex, strips it, and appends a “type confirm/cancel” footer.

Note (server tool misuse): `web_search_20250305` is an Anthropic **server-executed** tool. The manual “tool_use” fallback handling here sends back a fabricated `{"content": "Search completed."}` instead of real search data, which likely produces low-quality answers in that fallback path. **Note (broad exception swallowing):** The `MarketSurvey` lookup and the Claude-call retry both swallow all exceptions indiscriminately, masking real bugs with no logging. **Note (no input validation):** `message: str` has no max length — a large message inflates Claude token costs with no server-side guard.

`GET /suggestions/{plan_id}` and `GET /plan-versions/{plan_id}`

`business_chat.py:359-399` , `402-414` . Canned conversation-starter suggestions (no AI call) and the saved plan version history, respectively — both ownership-checked.

`_apply_update(plan, change_summary, user, db)`

`business_chat.py:465-519` . Saves the plan's current `plan_json` into `business_plan_versions` as a backup, calls Claude to rewrite the plan text, and overwrites `plan.plan_json` . **Note:** No optimistic-locking/concurrency control — two simultaneous chat sessions triggering an update will race, and the second `db.commit()` silently wins (lost-update). Also `current_plan[:2000]` truncation means a long existing plan gets silently cut off before being sent to Claude for rewriting.

business_dashboard.py

What this router is for: This router is the “launch and run your business” backbone: it lets a user convert an accepted `BusinessIdea` into an active `BusinessLaunchPlan` (generating an AI-written week-by-week launch plan via Claude), lists a user's plans, fetches a single plan's detail plus recent daily logs, and lets the user submit a `DailyBusinessLog` entry which triggers an AI-generated coaching response referencing that day's actual numbers.

`DailyLogRequest`

`business_dashboard.py:30-42` . `revenue` , `expenses` (float), `customers` (int) have **no validators** — negative values are accepted as-is.

Note: Nothing prevents negative revenue/expenses/customers, which corrupts aggregate calculations both here and in `business_chat.py` 's context builder.

`POST /start`

`business_dashboard.py:47-109` . Looks up the owned `BusinessIdea` (404 otherwise). If a plan already exists for that idea, returns it with `already_existed: True` (idempotent). Otherwise calls `_generate_launch_plan` (Claude, `claude-sonnet-4-6` , max 1500 tokens) and persists a new `BusinessLaunchPlan` . **Note:** If the Claude call fails inside `_generate_launch_plan` , the exception is caught *inside* the helper and an error string is returned and **saved as the plan's content** with a `200 OK` — the caller has no programmatic way to detect the failure, so a broken/expired API key silently produces a “successful” plan creation containing an error message as its body.

`GET /my-plans` , `GET /plan/{plan_id}`

`business_dashboard.py:114-133` , `138-193` . Straightforward ownership-filtered listing/detail queries; the detail endpoint defensively parses `plan_json` (handles real JSON, plain string, or unparseable text) and attaches the 30 most recent logs.

`POST /log`

`business_dashboard.py:198-236` . Verifies plan ownership, inserts a `DailyBusinessLog` , commits, then calls `_generate_coaching` (a second Claude call, 400 max tokens) and updates the same row with a second `db.commit()` . **Note:** Every daily log submission triggers a live Claude call — no caching/dedup, no rate limiting. **Note:** Two separate commits for one logical operation — a crash between them leaves a log with no coaching and no retry mechanism. **Note:** No validation that revenue/expenses/customers are non-negative.

`GET /logs/{plan_id}`

`business_dashboard.py:241-270` . Same ownership pattern, but returns **all** logs with no `LIMIT` — unlike `/plan/{plan_id}` (capped at 30), a long-running business returns unbounded history here.

`_generate_launch_plan` / `_generate_coaching`

`business_dashboard.py:275-303` , `306-337` . Both use bare `except Exception` with no logging (`import logging` is absent from this file entirely) — transient Claude API failures are indistinguishable from each other in production.

capital_access.py

What this router is for: This router (Sprint 17) powers KIP's “Capital Access Finder.” It ships a hardcoded catalogue of Zambian funding sources (CDF, CEEC, DBZ, ZANACO, FNB, BOZ, EMPRETEC), scores each source against a user's business profile with a simple heuristic, and optionally calls Claude to generate personalized funding advice.

FUNDING_SOURCES

`capital_access.py:20-239` . A hardcoded list of 7 dicts — static reference data, not DB-backed; any rate/program update requires a code deploy.

_score_match

`capital_access.py:244-273` . Computes a 0–100 “fit score”: +15/–20 for capital vs. `min_capital` ; +20 for a sector-tag substring match; penalties for CEEC/DBZ/ZANACO/FNB if operating <6 months; +20 bonus for the BOZ fund on rural-sounding locations. **Note:** The sector match (`s.lower() in sector_lower or sector_lower in s.lower()`) is a naive substring match, not semantic — “shop”/“boutique” won’t match “retail”.

GET /sources

`capital_access.py:304-307` . Returns the raw catalogue, unfiltered. **No authentication required** — intentional, since it’s static/non-sensitive, but worth noting since almost every other endpoint in the app requires login.

POST /match

`capital_access.py:310-384` . Requires auth. Pre-filters by capital sanity range, scores and sorts the top 5, and (if `ANTHROPIC_API_KEY` is set) calls Claude for 3 actionable funding tips. **Note:** `db: Session` is injected but never used in the function body — dead dependency, indicating leftover/incomplete code.

GET /letter-types

`capital_access.py:387-403` . Static list of 5 letter templates; no auth, no DB.

enhanced_logs.py

What this router is for: The backbone of KIP’s daily business-tracking feature (Sprint 6): generates a per-business custom log template, accepts daily operational/financial log submissions, runs ML forecasting (`ml_engine.run_all_predictions`) and Claude-generated coaching after each submission, aggregates weekly AI-narrated reviews, and stores/retrieves a detailed “market survey” per business plan.

GET /template/{plan_id}

`enhanced_logs.py:104-144` . Returns a cached `BusinessLogTemplate` if one exists, otherwise calls `generate_log_template` (Claude, falls back to a hardcoded default) and persists the result.

POST /submit

`enhanced_logs.py:149-272` . The core write endpoint: 1. Verifies plan ownership; prevents a duplicate log for “today.” 2. Computes `daily_profit` , `avg_sale` (divide-by-zero guarded), assigns `day_number` . 3. Inserts a new `EnhancedDailyLog` row (dict-typed fields JSON-serialized). 4. Reloads **all** historical logs and runs `run_all_predictions` (`ml_engine.py`), saving a new `MLPrediction` row. 5. Calls `generate_daily_coaching` (Claude) and stores the result with a second commit. 6. Flags `weekly_review_available` every 7th log.

Note (bug — dead docstring): `lang = get_language_from_request(request)` executes *before* the function’s triple-quoted string, so that string is no longer parsed as a docstring — harmless but likely unintended, and repeated in `trigger_weekly_review` . **Note (N+1/unbounded growth):** Every submission re-fetches **all** historical logs and re-runs `run_all_predictions` over the full history each time, with no cap — O(n) work per day of logging for long-running businesses. **Note (duplicate-check race):** The “already logged today” check-then-insert isn’t wrapped in a DB-level unique constraint/lock — two concurrent submissions in the same window could both pass the check and create two logs for the same day.

GET /history/{plan_id}

`enhanced_logs.py:277-295` . Returns up to `limit` most-recent logs (default 30). **Note:** `limit` is a client-controlled query param with no upper bound — `?limit=999999` forces a large unbounded response.

GET /predictions/{plan_id}

`enhanced_logs.py:298-324` . Returns the most recent `MLPrediction` row, scoped only by `MLPrediction.user_id` — it does not independently verify the parent plan belongs to the user, an inconsistent pattern vs. other endpoints that check the parent plan directly.

`POST /weekly-review/{plan_id}`

`enhanced_logs.py:329-393` . Fetches this week's and last week's logs (requires ≥ 3 to proceed), calls `generate_weekly_review` (Claude), persists a new `WeeklyReview` row. **Note:** Despite the docstring saying "Generate or retrieve," this endpoint always generates a brand-new review with no check for an existing one this week — calling it twice in the same week produces duplicate rows and duplicate Claude spend. No `try/except` around the Claude call either (unlike `generate_log_template`'s graceful fallback) — a failure here returns a bare 500.

`POST /survey`

`enhanced_logs.py:398-428` . Upsert-style endpoint for market survey data, keyed by `(user_id, plan_id)` . **Note (confirmed bug — NameError):** In the "update" branch, the code calls `save_survey_json(...)` referencing a variable `survey_obj` that is **never defined** in that function (the loaded/updated record is named `existing`). This raises `NameError` on every call to this endpoint when an existing survey is being **updated** — the update path of `/survey` is currently broken and always returns 500. The "create" branch is unaffected and works correctly.

`GET /survey/{plan_id}` , `GET /survey`

`enhanced_logs.py:431-443` , `446-455` . Return survey data for a plan, or all of the user's surveys across plans, as raw column dicts.

Helpers: `_log_to_dict` , `_get_logs_as_dicts`

`enhanced_logs.py:460-502` . **Note:** `_log_to_dict` passes `expense_breakdown` / `revenue_by_product` / `stock_movements` / `time_breakdown` straight through as raw JSON strings rather than `json.loads()` -ing them (unlike `get_predictions` , which does parse its JSON field) — an inconsistency that means consumers of `/history/{plan_id}` receive stringified JSON where they may expect parsed objects.

6. Backend Deep Dive — Enterprise, News & Templates Routes

This chapter covers `enterprise.py`, `news.py`, and `templates.py`.

enterprise.py

Mounted at `/api/enterprise`. Implements KIP's multi-branch "enterprise" feature: a Premium-tier account holder can create an `EnterpriseBusiness`, add `EnterpriseBranch` records (each linked to its own `BusinessLaunchPlan` dashboard and assigned to a separate KIP user as branch manager), receive in-app `BranchNotification`s, and get an LLM-generated cross-branch performance analysis. All endpoints require login; authorization beyond that is enforced ad hoc per-endpoint by comparing `owner_user_id` / `branch_manager_user_id` to `current_user.id`.

`_require_premium`

`enterprise.py:28-38`. Raises 403 unless `user.plan_tier` is `premium` / `enterprise` / `pro`. **Note:** This function's only call site (`create_enterprise`) has it **commented out**, so the premium gate is currently dead code — any free-tier user can create enterprises today, contradicting the docstring "Premium only."

`_ensure_tables`

`enterprise.py:41-82`. Runs raw `CREATE TABLE IF NOT EXISTS` SQL on nearly every request, duplicating the SQLAlchemy model definitions in `app/models/enterprise_models.py` — two sources of truth for schema that can drift.

`POST /create`

`enterprise.py:110-136`. Creates an `EnterpriseBusiness` owned by the caller (premium check disabled, see above).

`GET /my-enterprises`

`enterprise.py:139-166`. Lists owned enterprises with a live `branch_count`. **Note:** N+1 — one query per enterprise for its branch count.

`GET /enterprise/{enterprise_id}`

`enterprise.py:169-239`. Owner-only (404, not 403, for non-owned — a reasonable anti-enumeration choice). For each branch, looks up the manager, the linked plan, and the last 7 daily logs, computing 7-day totals. **Note:** Heavy N+1 — a 20-branch enterprise triggers ~60+ queries for one page load.

`POST /add-branch`

`enterprise.py:244-341`. Adds a branch, resolving the manager by email (404 if no account exists). Auto-generates a placeholder `BusinessLaunchPlan` for the branch, creates the `EnterpriseBranch` row, and notifies the manager. **Note:** `enterprise.branch_count` is recomputed by mixing a `.count()` query with a manual `+1`, worth manual verification/testing to confirm it doesn't double-count given the preceding `db.flush()`. **Note:** No validation that `manager_email` is well-formed.

`POST /update-target`, `GET /my-branches`, **notifications endpoints**

`enterprise.py:344-447`. Correct ownership-chain checks throughout (branch → enterprise → owner). `GET /my-branches` repeats the N+1 pattern (enterprise + owner lookup per branch).

`GET /enterprise/{enterprise_id}/kip-analysis`

`enterprise.py:452-523`. Owner-only. Pulls last 7 days of logs per branch (same N+1 pattern), and — if `ANTHROPIC_API_KEY` is set — sends a summary to `claude-sonnet-4-6` for a best/worst performer analysis. **Note:** The broad `except Exception as e: return {"analysis": f"Analysis error: {e}"}` exposes raw exception text to the client — could leak internal details depending on what the `anthropic` SDK raises.

news.py

Mounted at `/api/news` (shared with `briefings.py`). Serves live economic rates (USD/ZMW, fuel prices), a scraped news article feed, and a personalized "KIP Alerts" system matching new articles against each user's saved business ideas. Mixes public, unauthenticated endpoints (rates, articles) with user-scoped authenticated endpoints (alerts).

`GET /rates`

`news.py:21-38` . No auth. Returns the most recent `LiveRate` row per type, flagging `stale: True` if older than 6 hours.

`POST /rates/refresh`

`news.py:41-83` . **No authentication at all**, despite the docstring saying “Called by the scheduler or admin” — any anonymous caller can trigger external API calls and write new `LiveRate` rows. A potential abuse/cost vector.

`GET /articles`

`news.py:88-116` . No auth. Paginated, category-filterable. **Note:** `limit` / `offset` have no upper-bound validation — `limit=999999999` is accepted as-is.

`POST /articles/refresh`

`news.py:119-126` . **No authentication.** Schedules a `BackgroundTask` using the request-scoped `db` session. **Note:** Passing a request-scoped `Session` into a `BackgroundTask` that runs *after* the response is sent is a classic FastAPI pitfall — the session may already be closed by the time the task runs, causing intermittent errors depending on `get_db`’s exact teardown timing.

`_run_news_refresh` / `_generate_alerts_for_all_users`

`news.py:129-183` . Dedups scraped articles by URL (one `SELECT` per candidate article — $N+1$), then iterates **every** active user (another $N+1$, one query per user for their recent ideas) to generate alerts. **Note:** Scales linearly with total user count on every news refresh; could become slow as the user base grows, with no batching or async fan-out.

`GET /alerts`

`news.py:188-208` . **Note (docstring/behavior mismatch):** the docstring says “Return unread KIP alerts” but the query has no `is_read` filter — it returns all alerts, read or unread. The sibling `/alerts/count` endpoint correctly filters `is_read == False` .

`GET /alerts/count` , `PATCH /alerts/{id}/read` , `PATCH /alerts/read-all`

`news.py:211-249` . Correctly scoped and, for the bulk-mark-read case, implemented as a single efficient `UPDATE` rather than a loop.

templates.py

Mounted at `/api/templates` . A grab-bag covering: a static curated library of Zambian business-registration/funding form links, PDF business-plan generation, LLM-generated business letters, and a “general market survey” feature persisted both to SQLite and to per-town JSON files on disk with simple aggregation.

Module-level helpers

`templates.py:33-133` . `_slug()` sanitizes a location into a filesystem-safe filename; `_save_to_json()` loads/appends/rewrites a per-town JSON file; `_aggregate()` computes per-field averages and “most common” categorical values. **Note (race condition):** No file locking in `_save_to_json` — concurrent submissions for the same town can race and lose one submission’s data (read-modify-write race), a real risk on a multi-worker deployment. **Note:** `SURVEY_DIR` is created via `os.makedirs(..., exist_ok=True)` as a **module-import-time side effect** — unusual and fragile for a route file (fails hard at import time if the parent path isn’t writable).

TEMPLATES static list

`templates.py:138-152` . **Note:** Two of the URLs are Bing “click redirect” tracking URLs rather than direct links to the institutions — fragile and not ideal as permanent form links.

`GET /list` , `POST /generate-plan/{plan_id}`

`templates.py:155-245` . The latter assembles a PDF from plan JSON, best-effort survey data, and best-effort log summaries (three separate broad `try/except: pass` blocks), calling `generate_business_plan_pdf` in `business_plan_generator.py` . **Note:** The response filename is built from `plan.business_name.replace(' ', '_')` with no further sanitization — a `"` character in the business name could break the `Content-Disposition` header’s quoted filename value. **Note:** Internal exception text is passed straight to the client via `HTTPException(status_code=500, detail=str(e))` .

POST /generate-letter

`templates.py:258-293` . Requires auth + API key. Four hardcoded prompt templates keyed by `prompt_type` , defaulting silently to `introduction_letter` for an unrecognized type rather than erroring. **Note:** `recipient` , `amount` , `notes` are user-supplied free text interpolated directly into the LLM prompt with no sanitization — a prompt-injection surface, low real-world impact since the output is shown back to the same user.

GeneralSurveyRequest and POST /general-survey

`templates.py:298-388` . The schema's `field_validator` s coerce HTML-form string input into int/float, silently defaulting to `None` on failure (the file's own header calls this "Hotfix 7"). The submit endpoint dual-writes: a raw-SQL upsert (parameterized, safe) keyed `(user_id, location)` , plus the JSON-file path described above. **Note:** Because the SQL upsert key uses the raw, un-normalized `location` string while the JSON-file slug *does* normalize casing/whitespace, the SQLite table and the on-disk aggregation can drift out of sync for what a human would consider "the same town" (e.g. `"Lusaka"` vs `"lusaka "`).

GET /admin/towns , GET /admin/towns/{slug}

`templates.py:407-437` . **Note (missing admin check):** despite the `/admin/` path, these only require a valid logged-in user — no role/admin check exists. Any authenticated user can list and read all towns' raw survey data. **Note (path traversal risk):** `GET /admin/towns/{slug}` concatenates the raw `slug` path parameter directly into a filesystem path with no sanitization against `.. / /` sequences, unlike the write path (`_slug()`), which does sanitize. This should be fixed to reuse `_slug()` or reject non-alphanumeric characters before use.

GET /market-intelligence

`templates.py:439-465` . No auth required (intentionally public per its docstring) — aggregated-only data, safely sourced from real on-disk filenames rather than user input, so it does not share the path-traversal issue above.

7. Backend Deep Dive — Learning & BizSim Game

This chapter covers the three largest and most complex files in the backend: `learn.py` (772 lines), `bizsim_engine.py` (1,094 lines), and `bizsim_routes.py` (1,296 lines) — together comprising nearly 30% of the entire backend by line count.

learn.py — KIP Learn (financial literacy module)

What this router is for: Implements KIP Learn, a self-contained financial-literacy education module mounted at `/api/learn`. It ships three hard-coded courses (Financial Literacy 101, Financial Health 101, Starting a Business in Zambia) whose lesson text, quiz questions, and answer keys are Python literals in the module itself, not a database or CMS. It lets an authenticated user list courses, fetch a course's lessons, mark lessons complete, submit a scored final assessment per course, and view aggregate progress/badges. Progress and badge state persist to two auto-created tables (`learn_progress` , `user_badges`) via raw SQL, and a rule engine awards badges when a user scores $\geq 70\%$ on a course's final assessment, plus a meta-badge ("KIP Scholar") once all courses are passed.

Architecture: `learn.py:20-528` is the `COURSES` content catalog (hard-coded); `learn.py:530-535` is `BADGES` metadata; `538-594` are DB helpers; `597-772` are the five endpoints.

All five endpoints require authentication; there are no public routes in this file. **Note:** `import os` is never used — dead import.

`_ensure_tables`

`learn.py:540-566` . Runs `CREATE TABLE IF NOT EXISTS` on every request to every endpoint (5 call sites) — wasteful but functionally harmless. **Note:** The DDL is SQLite-specific (`AUTOINCREMENT`); if this app is ever run against Postgres, every Learn endpoint would 500 on first call.

`GET /courses`

`learn.py:610-638` . Returns all 3 courses with computed progress fields (`lesson_count` , `lessons_completed` , `progress_pct` , `badge_earned`). O(1) DB round-trips regardless of course count.

`GET /courses/{course_id}`

`learn.py:641-654` . Returns full course detail including lesson content and quizzes. **Note (business-logic / data leak):** Each lesson's `quiz` object — including `correct` (the correct-option index) and `explanation` — is returned verbatim in this response. Any authenticated user can read every lesson quiz's answer key before attempting it, undermining the quizzes as any kind of check.

`POST /complete-lesson`

`learn.py:657-670` . Idempotent upsert marking a lesson complete. **Note (missing validation):** Neither `course_id` nor `lesson_id` is checked against the `COURSES` catalog — a caller can mark an arbitrary/nonexistent id as "completed," polluting `_get_progress`'s output invisibly.

`POST /submit-assessment`

`learn.py:673-757` . Scores a submitted assessment (`correct/total * 100` , 70% pass threshold), persists the result (only the most recent attempt is kept — no history), and awards the course badge on a pass. Also checks (via an N+1 query per course) whether all courses are passed to award the "KIP Scholar" meta-badge. **Note (missing validation / crash risk):** `req.answers` values are never bounds-checked against each question's option count. An out-of-range positive index raises an unhandled `IndexError` → unhandled 500; a negative index is valid Python indexing and silently returns the *last* option, producing confusing, non-obvious scoring for adversarial input. **Note:** No re-attempt limiting/cooldown — a user can resubmit repeatedly to game a badge.

`GET /my-progress`

`learn.py:760-772` . Aggregate summary; inherits the "phantom course" risk from `complete-lesson`'s missing validation.

bizzsim_engine.py — BizSim simulation logic

What this file is for: The pure simulation/logic layer for KIP's "Business Simulation" game (per the design doc [KIP_BizSim_GameSpec.md](#), a 30-day, story-driven survival game across six Zambian small-business types, each with an inherited debt and a named AI rival). This file has **no** `@router` endpoints — `bizzsim_routes.py` imports and orchestrates it. Nearly everything here is a stateless function (input → computed output); only `record_transaction` and `get_balance_sheet` touch the database.

Architecture: business catalogue ([16-136](#)), stories/debt/rivals ([138-271](#)), chart of accounts ([273-289](#)), demand model ([292-381](#)), unique per-business mechanics ([384-428](#)), double-entry ledger ([431-493](#)), market state & macro-event calendar ([496-533](#)), AI-competitor simulation ([536-569](#)), event-card system ([572-902](#)), phase-unlock logic ([905-934](#)), missions ([937-988](#)), AI coaching/Claude integration ([991-1094](#)).

`BUSINESS_TYPES` and `BUSINESS_STORIES`

`bizzsim_engine.py:16-271`. Static config (economics, mechanics) and narrative (backstory, debt terms, rival persona) per business type — every other function reads from these dicts. **Note:** No unknown-key guard anywhere — an invalid `biz_type` raises an uncaught `KeyError` rather than a clean 400. **Note:** `debt_schedule` entries are inconsistently shaped — grocery/transport/food_stall/phone include an `"amount"` key, poultry/restaurant do not; code reading `entry["amount"]` without a default would `KeyError` for those two.

`calculate_demand`

`bizzsim_engine.py:294-381`. A hand-tuned multiplicative demand model: `demand = base × price_factor × marketing_factor × market_factor × competitor_factor × special_factor + noise`. Marketing uses logarithmic diminishing returns; noise is Gaussian. **Note:** Uses the unseeded module-level `random`, unlike `select_event_for_day` (which deliberately seeds per session+day) — demand outcomes are non-reproducible/non-replayable. **Note:** The `price_elasticity` field defined per business type in `BUSINESS_TYPES` is never referenced anywhere in this function — dead config.

`apply_unique_mechanics`

`bizzsim_engine.py:386-428`. Applies each business type's signature risk after a day's sales: grocery spoilage, poultry mortality, transport breakdowns (can zero out revenue that day), phone FX volatility. **Note:** Restaurant and food_stall have config fields (`waste_rate`, `daily_prep_waste`) but **no handling branch exists in this function at all** — either dead config or an incomplete feature.

`record_transaction` / `get_balance_sheet`

`bizzsim_engine.py:433-493`. Real double-entry bookkeeping: raises `ValueError("Ledger imbalance!")` before writing anything if debits ≠ credits. **Note:** No validation that an entry's account name is a known key in `ACCOUNTS` — a typo'd account name is written silently and excluded from the balance-sheet totals with no error.

`MACRO_EVENTS` / `get_market_state`

`bizzsim_engine.py:498-533`. A fixed, hardcoded calendar of 11 story-wide events (one per specific day). Every session sees the identical macro event on the same day. **Note:** The exponential "drift back to normal" smoothing only applies to `demand_index` / `weather_factor` — `fuel_price` / `usd_zmw_rate` / `cogs_mult` set by an event persist unchanged indefinitely, which may be intentional or an oversight.

`simulate_competitor_day`

`bizzsim_engine.py:538-569`. Picks a rival's price by personality, but computes competitor demand as a flat 60%-of-base random draw **independent of the competitor's own price** — a simplification, not a fully simulated market.

Event-card system

`bizzsim_engine.py:572-902`. ~24 random event cards plus 2 guaranteed narrative beats (day-15/day-29 debt reminders). `event_card_payload` strips all consequence fields before sending to the client — the security boundary that keeps the frontend from ever seeing (and tampering with) numeric outcomes ahead of a player's choice. `select_event_for_day` uses a **deterministically seeded** `random.Random(f"{session_id}-{day}")`, making repeated calls idempotent. **Note:** Within a single day's event-selection loop, already-picked events aren't excluded — the same card could theoretically be selected twice on a two-event day.

`check_phase_unlock`, `MISSIONS` / `check_missions`

`bizzsim_engine.py:907-988`. Pure gate/rule functions. **Note:** `check_missions` wraps each mission's `check` lambda in `except Exception: pass` — a bug in one mission's check silently disables it forever with no logging.

AI coaching / Claude integration

`bizsim_engine.py:991-1094` . Three functions (`generate_ai_coaching` , `generate_market_event` , `ai_supplier_negotiation`), each building a prompt, calling `claude-sonnet-4-6` , and falling back to templated text on any exception. **Note (applies to all three):** Declared `async def` but call the **synchronous** `anthropic.Anthropic` client directly — a blocking network call inside an async function stalls the entire FastAPI event loop for the duration of the API round-trip, not just the current request. **Note:** A new `anthropic.Anthropic(...)` client is constructed on every call rather than reused. **Note:** `generate_market_event` 's JSON extraction (`text.find("{")` / `text.rfind("}")`) is a fragile way to isolate a JSON object from free-text LLM output.

bizsim_routes.py — BizSim API endpoints

What this router is for: The FastAPI endpoints for the BizSim game, mounted at `/api/bizsim`. Owns persistence (creating/migrating its own SQLite tables), advances the simulation one day at a time, maintains the double-entry ledger, and layers in Claude-powered features (AI loan officer, supplier negotiation, customer haggling, weekly coaching, AI-generated market events). Almost all simulation math lives in `bizsim_engine.py`; this file is mostly request validation, session loading via raw parameterized SQL, and orchestration.

SQL injection check: every `db.execute(text(...))` call binds user-derived values through named parameters. The one f-string-built SQL statement (`bizsim_routes.py:108`, a migration `ALTER TABLE`) only interpolates a hardcoded internal column list, never user input. **No SQL injection vector was found in this file.**

POST /start

`bizsim_routes.py:272-366`. Validates `biz_type`, hard-deletes any prior session (regardless of status) plus its ledger/messages/event-log rows, creates a fresh session pre-populated from `BUSINESS_STORIES`, and posts opening ledger transactions. **Note:** Deleting-then-recreating on restart destroys a player's entire game history rather than archiving it. `req.biz_name` has no length limit and later flows into AI-prompt contexts (low-severity prompt-injection surface, since it only affects the player's own game).

GET /event/today , POST /event/choose

`bizsim_routes.py:383-484`. Deterministically-seeded event selection (idempotent re-fetch); choosing resolves the event, applies clamped reputation deltas and a real ledger-posted cash delta. **Note:** A positive `cash_delta` (e.g. a grant) is booked as debit-cash/credit-**expense** rather than to an income account — balances still reconcile, but such windfalls never appear as "income" in the balance sheet the way sales do.

POST /day/run — the core simulation loop

`bizsim_routes.py:487-771`. Advances the game one day: updates market state, rolls a chance of an AI-generated event, folds in any resolved event-card consequence, resolves effective COGS (including any negotiated discount), validates price, computes stock purchase, calls `calculate_demand`, applies unique business mechanics, computes financials, posts ledger transactions, checks missions/phase-unlock, persists, and (every 7th day) generates AI weekly coaching.

Note (exploitable — missing input validation): `DayRequest.stock_buy`, `marketing`, and `save_amount` all lack `ge=0` Pydantic constraints, and the handler's clamping only bounds them from *above*. A negative `stock_buy` produces a negative cost that **adds** cash rather than removing it — a player can inflate their cash balance arbitrarily. A sufficiently negative `marketing` (`≤ -100`) crashes `calculate_demand`'s `math.log1p` call with an unhandled `ValueError` (500) rather than a clean 400.

Note (dead fields): `req.take_loan` and `req.hire_employee` are accepted by the schema but never read anywhere in this router — there is no `/hire` endpoint, so `hire_employee: true` has no effect despite the `employees` / `employee_cost` columns existing and being read elsewhere.

Note (ledger/cash desync bug): the loan-interest ledger posting credits `cash` by `interest × 2` (double-counted) while the session's real `cash` column is only debited `interest` once, and separately debits (reduces) `loan_payable` for interest accrual that never increases the session's actual `loan_balance`. Over time, the ledger-derived balance sheet drifts from the authoritative `bizsim_sessions.cash` / `loan_balance` columns — the two "cash" figures shown to the frontend will disagree.

Note (narrative debt never enforced): each business's story promises real stakes ("repay K8,000 by day 30 or lose the stall"), but nothing in this router ever deducts `debt_balance`, checks it against `debt_due_day`, or penalizes a missed deadline — there's also no `/debt/repay` endpoint. This looks like an incomplete feature, not a bug, but the in-game narrative currently promises consequences the backend doesn't deliver.

GET /statements

`bizsim_routes.py:774-827`. Builds P&L/cash-flow from the session's in-memory `history` array (capped to the last 60 days by `/day/run`), while `balance_sheet` is ledger-derived (full history) — for games past day 60, these two data sources visibly disagree. **Note:** Inconsistent response key naming between the "no history yet" branch (`cash_flow`) and the populated branch (`cash_flow_chart`).

POST /customer-chat , POST /loan/apply , POST /negotiate

`bizsim_routes.py:830-1216`. All three derive game outcomes (`agreed_price`, loan approval, negotiated COGS) by regex-parsing free-text LLM output rather than structured/validated output, falling back to a canned response on any API exception. **Note (/loan/apply):** `LoanRequest.amount` has no `gt=0` validation; combined with a permissive fallback-approval rule, a zero/negative loan amount is not

server-blocked. **Note (/negotiate):** A real server-side floor (negotiated COGS cannot go below 60% of standard cost) is a good defensive guard against the LLM being talked into an unrealistic price. However, this endpoint duplicates the Anthropic-calling logic inline rather than reusing `ai_supplier_negotiation` (imported but unused) from `bizsim_engine.py`.

`POST /loan/repay`

`bizsim_routes.py:1068-1116`. The most completely validated mutating endpoint in the file (checks positivity, affordability, and loan existence) — a good reference point for what the other endpoints are missing.

`GET /leaderboard`

`bizsim_routes.py:1230-1258`. **Note (potential PII exposure):** the only data-returning endpoint with no authentication requirement, and it returns each entrant's real `full_name` publicly. Worth confirming with product/privacy whether this is an intended public leaderboard or should use a display alias.

Cross-cutting summary for BizSim

No SQL injection risk was found anywhere in `bizsim_routes.py`. The main risk surface is **missing lower-bound input validation** on `/day/run` and `/loan/apply` (cash-inflation exploits and a potential unhandled 500), plus several **data-integrity drift bugs** between the double-entry ledger and the session's plain columns (loan interest, narrative debt never enforced) that will produce visibly inconsistent numbers to players over a long game.

8. Backend Deep Dive — The AI Engine (services/)

This chapter documents the six files in `backend/app/services/` that form KIP’s core business-idea generation pipeline: prompt construction, the Claude API call itself, RAG retrieval, scoring, ML forecasting, and news aggregation.

kip_engine.py

What this file does and where it fits in the pipeline: `kip_engine.py` is the orchestrator of the whole K-BIG-1 (“Kwacha Business Idea Generator”) engine — the single function the chat route calls (`generate_kip_response`) to turn a user’s chat message into a KIP reply. It extracts context clues (location, capital) from the conversation, pulls relevant knowledge from the ChromaDB RAG store (`knowledge_base.py`), pulls town/location intelligence, pulls the user’s idea history from the database, assembles the final system prompt (via `kip_prompt.py`), calls the Anthropic Claude API (with the `web_search` server-side tool enabled), parses the response, detects whether the reply is a “formal business recommendation,” and persists that recommendation to the database.

Idea detection

`kip_engine.py:19-65` . `IDEA_HEADER / IDEA_VERIFICATION_KEYWORDS` mirror the exact header string mandated in `kip_prompt.py`’s system prompt. `is_business_idea(text)` gates detection on length + header + keyword match — pure string/keyword matching on raw LLM text output, with no structured-output/JSON-schema enforcement, so it’s inherently fragile. `_extract_idea_name` is a small heuristic state machine parsing loosely-structured LLM output.

`_extract_location / _extract_capital / _extract_factors`

`kip_engine.py:68-99` . Regex/dictionary heuristics pulling a Zambian town name and a capital amount out of free text, scanning only the last 6 messages of history. **Note:** Assumes `m.content` is always a usable string; if any historical message content is `None`, string concatenation raises `TypeError` .

generate_kip_response(...) — the function that talks to Claude

`kip_engine.py:116-264` . Key facts:

- ANTHROPIC_API_KEY handling** (`134-142`): read fresh via `os.getenv` on every call. If unset (or still the `.env.example` placeholder), returns a canned message and skips the API call entirely — no exception, no log. Good for local dev; means a misconfigured production deployment degrades silently rather than alerting anyone.
- Context assembly** (`144-177`): RAG search, town profile, last 10 saved ideas for this user (wrapped in a bare `try/except: pass` — any DB error here is silently swallowed).
- Web search tool** (`186-192`): `tools = [{"type": "web_search_20250305", "name": "web_search"}]` — Anthropic’s server-side web-search tool; the backend implements no search logic of its own. **Note:** Uses the basic `web_search_20250305` tool type rather than the current dynamic-filtering variant (`web_search_20260209` , available on Sonnet 4.6+), which would likely reduce token usage with no other code changes.
- Client instantiation** (`194`): a new `anthropic.Anthropic(api_key=api_key)` is constructed on every single call rather than reused.
- The API call** (`196-215`): `model="claude-sonnet-4-6"` (hardcoded literal, repeated **three separate times** in this one file — lines 198, 209, 244), `max_tokens=2048`, no `temperature` override, no `system` caching, **no streaming**. **Note (error handling gap):** Only `anthropic.BadRequestError` is caught (and only to detect the web-search-unsupported case via fragile string matching on the error message). `RateLimitError` , `APIConnectionError` , `AuthenticationError` , `InternalServerError` , `APITimeoutError` are all **unhandled** and will propagate up, presumably surfacing as an unstyled 500 to the end user. **Note (async correctness):** This function is `async def`, but every I/O call inside it — Claude, ChromaDB, the DB query — is synchronous/blocking. None is awaited or run in a thread pool; a Claude round-trip blocks the FastAPI event loop for its full duration.
- Manual “tool result” loop** (`225-250`): builds fake `tool_result` blocks with static content `"Search completed."` for every `tool_use` block and re-calls the API — this is client-side tool-result semantics applied to a **server-side** tool that doesn’t need it, and is likely dead/vestigial code that would confuse the model if actually exercised.
- _save_idea** (`267-318`): dedups by exact `idea_name` string match (near-duplicate names like “Chikondi’s Salon” vs “Chikondi Salon” both get saved), and dynamically inspects the `BusinessIdea` model’s columns so it’s defensive against schema drift.

ANTHROPIC_API_KEY usage summary (kip_engine.py):

Aspect	Detail
Where read	<code>os.getenv("ANTHROPIC_API_KEY", "")</code> , freshly on every call

Aspect	Detail
Model ID	"claude-sonnet-4-6" — hardcoded, repeated 3× in this file alone
Missing-key behavior	Graceful canned response, no exception
Retry/backoff	Only the Anthropic SDK's built-in default; no app-level retry
Explicit error handling	Only <code>BadRequestError</code> , for the web-search-unsupported case
Streaming	Not used
Async correctness	<code>async def</code> , but every call inside is blocking

Note: No user input is escaped before being interpolated into the system prompt via `idea_history` (built from prior `BusinessIdea.idea_name` values). A specially crafted idea name saved in a prior turn could inject instructions into a later system prompt — low severity since it only affects the acting user's own session data.

kip_prompt.py

What this file does: Contains no API calls — a pure prompt-template module defining KIP's entire persona, domain knowledge, and output-format rules as Python string constants, exposed via one function, `build_system_prompt()`.

- `KIP_IDENTITY` (7-25): persona as "Zambia's first AI-powered business intelligence advisor."
- `KIP_ZAMBIA_CONSTANTS` (27-50): hardcoded Zambia-specific economic facts (financing sources and loan ranges, PACRA/ZRA costs, macro context) — a hand-curated knowledge base baked directly into the prompt, supplementing (and able to go stale independently of) the ChromaDB RAG content.
- `KIP_BPRA_PROCESS` (52-79): a 6-step reasoning process and the literal header string `## 📌 KIP BUSINESS RECOMMENDATION` that `kip_engine.py`'s detector depends on — a direct example of prompt-engineering-as-application-logic: idea detection only works because this prompt instructs the model to emit an exact, predictable header, and LLMs are not 100% reliable at following exact formatting.
- `KIP_OUTPUT_FORMAT`, `KIP_PERSONALITY` (81-139): the literal markdown template and tone rules.
- `build_system_prompt(...)` (142-176): concatenates the five static sections, then conditionally appends RAG-retrieved knowledge, town profile, and idea history only if each is non-empty.

Note (prompt-caching opportunity): The entire static portion of the system prompt (identity + constants + process + format + personality) is byte-identical across every request from every user, and is rebuilt as fresh string concatenation on every call. This is an ideal `cache_control: {type: "ephemeral"}` breakpoint candidate — but no `cache_control` is applied anywhere in `kip_engine.py`'s API call, so every request pays full input-token price for this multi-hundred-token static prompt.

Note: All loan amounts/tax rates/fees in `KIP_ZAMBIA_CONSTANTS` are hardcoded with no "last updated" marker or versioning — if ZRA's turnover-tax threshold or PACRA's fees change, this file must be manually edited.

kip_score.py

What this file does: Computes the user-facing "KIP Score" (0–100 gamification metric shown on the dashboard). Makes **no calls to Claude or any LLM** — pure SQL aggregation and arithmetic.

`calculate_kip_score(user_id, db)` (13-47) counts conversations/ideas/accepted-ideas/plans/logs, applies a weighted point allocation (conversations 4pt/cap 20, ideas 10pt/cap 20, accepted-ideas 20pt/cap 20, plans 25pt/cap 25, logs 1pt/cap 15 — max 100), and maps the total onto one of five named bands (Explorer → Researcher → Planner → Entrepreneur → Operator).

Note: Inconsistent error handling — the `BusinessLaunchPlan` / `DailyBusinessLog` count queries are wrapped in `try/except: default to 0`, but the `Conversation` / `BusinessIdea` queries are not, so a database hiccup produces different failure behavior (silent zero vs. hard crash) depending on which table has the problem.

knowledge_base.py

What this file does: KIP's **RAG (Retrieval-Augmented Generation) layer** — wraps a persistent ChromaDB vector database and exposes a `search()` method that `kip_engine.py` calls before every Claude request to fetch semantically relevant text chunks for injection into the system prompt.

`get_knowledge_base()` (16-45) is a manual singleton/lazy-init using module-level globals: opens (or creates) a `chromadb.PersistentClient`, and — on first creation — configures a `SentenceTransformerEmbeddingFunction` using `sentence-transformers/all-MiniLM-L6-v2`, a locally-run open-source embedding model (Anthropic has no first-party embeddings endpoint). This is the classic RAG pattern: rather than cramming Zambia-specific knowledge into the static prompt, an arbitrarily large corpus can be searched semantically and only the top-K most relevant chunks pulled into context per request.

`search(query, top_k=4)` (52-64) embeds the query, does approximate-nearest-neighbor search, and joins results with a `"\n\n---\n\n"` separator — this string flows directly into `kip_prompt.py`'s `retrieved_knowledge` parameter.

Note (silent degradation, no observability): Both `get_knowledge_base()` and `search()` use bare `except Exception` blocks that `print()` and continue — if the RAG layer breaks (ChromaDB corruption, disk issue, dependency problem), the app silently runs with **zero grounding** indefinitely, with no structured logging, metrics, or alerting to notify an operator.

Note: The embedding model downloads weights from Hugging Face Hub on first use if not already cached — an implicit external dependency that would fail in a fully network-isolated deployment unless pre-cached.

ml_engine.py

What this file does: Not part of the Claude/idea-generation pipeline at all — a self-contained analytics module (Sprint 6) used once a user has an active business submitting daily logs. Uses scikit-learn (with hand-rolled fallbacks if unavailable) for revenue forecasting, trend/anomaly detection, and break-even tracking.

- `forecast_revenue_7d` (36-113): the core “ML” function. With <5 logs, returns a flat average (confidence 0.3). With scikit-learn available, fits a `LinearRegression` on `[day_index, sin(day_of_week), cos(day_of_week)] → revenue` — a standard cyclical/sinusoidal encoding so Sunday and Monday are treated as “close” rather than maximally distant. Confidence is derived from a manually-computed R^2 , rescaled into a `[0.2, 0.95]` heuristic band (not a rigorous statistical interval).
- `detect_customer_trend`, `detect_peak_day`, `detect_expense_anomaly`, `predict_stockout`: threshold-based, rule-based classifiers — none of these are ML in the statistical-model sense, despite being grouped with the regression-based forecast.
- `calculate_breakeven_progress` (266-301): extrapolates using a hardcoded 2%-daily-growth-rate assumption not derived from the actual data trend.

Note: `calculate_breakeven_progress`'s days-to-target formula divides by an arbitrary `avg_daily * 0.02` — a heuristic dashboard number that should probably be labeled as an estimate, not a rigorous projection, in the UI.

news_service.py

What this file does: Also makes **no calls to Claude** — a standalone RSS-scraping and rule-based news-categorization/alerting service: fetches Zambian news RSS feeds, keyword-categorizes articles, matches categories against each user's saved business ideas, and generates template-based alert messages.

Note (real bug): `fetch_fuel_prices()` (120-140) scrapes `zera.co.zw` — a **Zimbabwean** domain — when the intent (per the ZERA-Zambia comment and a commented-out `.org.zm` URL directly above it) is clearly Zambia's fuel regulator. This looks like a genuine typo that would silently produce incorrect or empty fuel-price data.

Note: `fetch_copper_price()` is a non-functional stub that always returns `None`, despite `kip_prompt.py`'s `KIP_ZAMBIA_CONSTANTS` calling out copper as ~70% of Zambia's export earnings — a real gap between the prompt's stated priorities and available live data.

Note: `_parse_rss` uses regex against raw XML/HTML instead of a proper parser (`xml.etree.ElementTree` / `feedparser`) — fragile to any feed-format variation, with silent (not raised) failure on mis-extraction.

Note: `scrape_news()` runs its 5 source fetches (up to 20s timeout each) fully sequentially with a `time.sleep(0.5)` “polite delay” between each — a full scrape can take tens of seconds, and if this were ever invoked in a request-response path rather than a background job, it would block the event loop for that entire duration.

Note: `_compose_alert_message`'s docstring implies KIP “writes” a personalized message, but the implementation is static Python dict template substitution, not an LLM call — worth knowing when reading the code, since the naming is misleading.

AI Pipeline — end-to-end narrative

A business-idea request begins when the chat route calls `generate_kip_response()` in `kip_engine.py:116`. The function first checks `ANTHROPIC_API_KEY` (134-142); if missing, it returns a canned message and stops. Otherwise it extracts location/capital hints from the conversation (91-99 , 144), queries the ChromaDB RAG store via `knowledge_base.py`'s `get_knowledge_base().search(user_message, top_k=4)` (147-148), which embeds the query with a local `sentence-transformers/all-MiniLM-L6-v2` model and returns the top-4

semantically similar knowledge chunks. It also fetches town/location context and, from the database, the user's last 10 saved ideas so KIP avoids repeating itself (158-171). All of this is passed into `build_system_prompt()` in `kip_prompt.py:142-176` , which concatenates KIP's static persona/rules/output-format sections with the dynamic retrieved content into one system prompt string. Back in `kip_engine.py:194-203` , a fresh `anthropic.Anthropic` client is created and `client.messages.create()` is called with model `claude-sonnet-4-6` , `max_tokens=2048` , the assembled system prompt, the web-search server tool, and the last 12 turns of conversation history. Claude may autonomously trigger a server-side web search before composing its answer. The reply text is extracted (102-113), checked against `is_business_idea()` (32-38) by looking for the exact `## 📌 KIP BUSINESS RECOMMENDATION` header the prompt mandated, and if matched, `_save_idea()` persists it to the `BusinessIdea` table (267-318). `kip_score.py` and `news_service.py / ml_engine.py` operate independently downstream of this saved data, powering the dashboard's engagement score and news alerts respectively — neither calls Claude.

9. Backend Deep Dive — Supporting Services

This chapter covers the remaining nine files in `backend/app/services/`: briefing generation, PDF business-plan generation, the business dashboard's AI helpers, transactional email, daily-log coaching, map intelligence, subscriptions/billing, survey persistence, and translation.

briefing_service.py

What this file does: Generates KIP's daily "Economic Briefing" — a curated digest of Zambian economic news translated into plain language. Calls Claude with the `web_search` tool to find recent stories from a fixed list of trusted Zambian sources, asks it to simplify each story and add business-impact commentary, parses the resulting JSON, and upserts it into `EconomicBriefing`. Meant to run once a day via a scheduler, plus on-demand via the `/refresh` endpoint (Chapter 4).

A single module-level Anthropic client is created at import time (42); credentials come from `ANTHROPIC_API_KEY`. `generate_briefings()` (131-202) calls `claude-sonnet-4-6` with `max_tokens=8000` and the `web_search_20250305` tool, strips markdown fences with regex, and `json.loads()` s the array. `store_briefings()` (205-251) deletes any rows already stored for today before inserting the fresh batch (so a manual refresh replaces rather than duplicates), with each item's insert individually try/excepted so one malformed item doesn't abort the batch.

Note: `PRIORITY_SOURCES` (46-57) is defined but **never referenced anywhere in the file** — the actual source list Claude sees is a separately hand-maintained copy embedded in the system prompt constant. Dead code and a duplication hazard: updating one list without the other silently desyncs them. **Note:** No handling of `stop_reason == "pause_turn"` on the web-search tool call (unlike the partial handling in `kip_engine.py`) — a long multi-query search session could be cut short with no retry, silently returning zero briefings for the day. **Note:** `store_briefings`'s `.get("field", default)[:N]` pattern breaks if Claude explicitly returns `null` for an optional field (valid under the documented schema) — `None[:500]` raises `TypeError`, caught per-item so that story is dropped rather than stored with an empty string.

business_plan_generator.py

What this file does: Produces a bank-ready PDF business plan (for CDF/CEEC/DBZ/commercial-bank submission). Uses Claude to generate the plan's prose content as structured JSON, then assembles it into a styled multi-page PDF via ReportLab, with a KIP-branded cover page, headers/footers, KPI tables, and financial-projection tables.

`_generate_plan_content(...)` (196-294) calls `claude-sonnet-4-6`, `max_tokens=3000`, with **no system parameter** (all instructions folded into one user message, unlike `kip_engine.py`), requesting exactly 8 JSON sections plus startup-cost/projection arrays. On any exception, falls back to `_fallback_content(...)` (296-366), a hand-written template using literal placeholder amounts like `"K0,000"`.

Note: `_logo_data()` (46-51) hardcodes two absolute **Unix sandbox paths** (`/home/claude/kip_logo_v2.png`, etc.) that will never exist on a normal Windows/Linux production deployment — the cover-page logo silently never renders in production. Should reference an app-relative `assets/` path (as `email_service.py` correctly does) or an env var. **Note:** No error handling around the final `doc.build(...)` call — if any AI-generated text contains an unescaped `<`, `>`, or `&` (plausible in financial prose, e.g. "profit < K5,000"), ReportLab's mini-HTML `Paragraph` markup parser can raise an unhandled exception, since AI text is never HTML-escaped before being wrapped in `Paragraph(...)`. **Note:** Uses `print()` for errors instead of the `logging` module used elsewhere in the codebase (e.g. `briefing_service.py`), and its markdown-fence-stripping logic is implemented differently (and could drift out of sync) from the near-identical logic in `dashboard_service.py`.

dashboard_service.py

What this file does: Powers the Business Dashboard (Sprint 4): generates a detailed week-by-week launch plan via Claude, produces daily-coaching messages from a user's logs, and contains a rule-based (non-AI) profitability check.

`generate_launch_plan(...)` (122-176) calls `claude-sonnet-4-6`, `max_tokens=3000`, with a system prompt hardcoding Zambian specifics (PACRA ~K120-150, ZRA turnover threshold, ZANACO/FNB, MTN MoMo/Airtel Money). `generate_daily_coaching(...)` (256-306) formats up to the last 14 logs and asks for a fixed emoji-headed **plain-text** format (not JSON), capped at 350 words "because this is read on a phone after a long day."

Note (dead code / logic bug): `check_profitability` (346-393) requires `len(logs) >= 30` to run at all, but then has an `if len(logs) >= 28:` branch (already always true at that point) versus an `else` branch meant for "only one 14-day window available" — that `else` branch is **unreachable dead code**, since 30+ logs always satisfies the `>= 28` check. This looks like a threshold mismatch (probably meant `>= 14` and `>= 28`, not `>= 30` and `>= 28`). **Note:** Both Claude-calling functions check for the literal placeholder string `"your-anthropic-api-`

`key-here` as a stand-in for “not configured” — if that example value in `.env.example` ever changes, this check silently stops working and the app attempts (and fails) a real API call instead of gracefully falling back. **Note:** Uses `print()` rather than `logging`, same inconsistency as `business_plan_generator.py`.

email_service.py

What this file does: A small, provider-agnostic **SMTP** email helper (currently just the account-verification email). Builds a multipart MIME message (plain-text + HTML, with an optional inline logo) and sends via `smtplib` over STARTTLS. If SMTP isn’t configured, it **logs the email instead of sending it**, so local dev never breaks on a missing mail setup.

All configuration (`SMTP_HOST` , `SMTP_PORT` default 587, `SMTP_USER` , `SMTP_PASSWORD` , `FROM_EMAIL` , `FROM_NAME` , `FRONTEND_URL`) comes from environment variables with safe empty-string defaults — **no hardcoded credentials anywhere in this file**, and the logo path is resolved relative to the file’s own location (a more portable pattern than `business_plan_generator.py`’s hardcoded Unix paths). `_send_email()` (`35-70`) never raises — all failures are caught and logged, returning `False`.

Note: The SMTP-not-configured fallback path logs the **full plaintext email body — including the verification token embedded in the link — at WARNING level**. If SMTP is ever accidentally left unconfigured in a production environment with broader log retention than the app’s data-retention policy, this could leak verification tokens into logs. **Note:** Uses `smtplib.SMTP(...).starttls(...)` unconditionally — no support for an implicit-TLS provider on port 465 (`smtplib.SMTP_SSL`) if that’s ever needed.

log_service.py

What this file does: The AI-coaching layer behind the enhanced daily-logging feature (Chapter 5): builds a custom daily-log template for a new business, turns a day’s log entry plus ML predictions into a coaching message, and rolls up a week of logs into KPIs and a narrative weekly review — every AI call has a hand-written fallback.

Note (confirmed bug — NameError): In `_generate_review_narrative`’s exception fallback (`log_service.py:356`), the code references an undefined variable `kips` instead of `kpis`. Since this line only executes when the Claude call has *already* failed (inside `except Exception:`), and `generate_weekly_review` has no surrounding try/except around this helper, a Claude API failure during weekly review generation would crash the request with an unrelated `NameError` instead of degrading gracefully as the fallback path intends. **Note:** `generate_daily_coaching`’s API-key check is only `if not api_key:` — unlike `generate_log_template`, it doesn’t also check for the `.env.example` placeholder string, so a leftover placeholder key triggers a live (failing) API call each time rather than skipping straight to the fallback (still safe, since the failure is caught, but inconsistent). **Note:** All three AI functions print errors to stdout rather than using a logger.

map_service.py

What this file does: Provides “map intelligence” by querying OpenStreetMap’s free Overpass API for businesses near a Zambian town, classifying them into KIP categories, scoring market saturation, and identifying underserved gaps — then caching the result as JSON on disk so runtime chat requests never hit the network. A companion (not-included) script `fetch_maps.py` is meant to pre-warm the cache for major towns.

`get_map_context(location_string)` (`205-224`) — the function actually used at chat time by `kip_engine.py` — resolves a location string via `resolve_location()` (from `town_coordinates.py`), tries the exact cache key, then falls back to the “parent city” (first word of a multi-word key). Returns `""` if nothing is cached, so callers can safely concatenate it into a prompt with no special-casing.

Note: `REQUEST_DELAY = 1.2` (a “politeness delay” constant) is defined but **never actually used anywhere** in the file — only a 30s sleep on an explicit 429 response exists. **Note:** This service depends entirely on a pre-populated on-disk cache — if a location was never pre-fetched, `get_map_context` silently returns nothing rather than fetching live data at request time (by design, for zero conversational latency, but there’s no expiry/refresh logic visible in this file, so stale caches persist indefinitely).

subscription_service.py

What this file does: Implements KIP’s free/premium tiering: resolving a user’s effective plan tier (accounting for trial windows and expiry), enforcing daily usage limits for free users, activating 30-day trials, and activating/cancelling paid premium subscriptions once a payment provider confirms payment.

`get_effective_tier(user, db)` (`25-53`) is the core resolution function — checks trial/period expiry and downgrades to free automatically when a window lapses. `check_idea_limit` / `check_chat_limit` (`115-134` , `147-163`) implement the free-tier daily caps (3 ideas/5 chats, +2 “soft” overage before hard blocking) — see Chapter 11 for why these currently have no effect in production.

Note (bug): `cancel_premium` (244-249) only flips `status` to "cancelled" and does **not** clear `plan_tier` / `current_period_end` . The docstring claims the user "retains access until period end," but `get_effective_tier` 's premium-period check is gated on `status == "active"` — once cancelled, that branch no longer matches, so a cancelled user actually drops to "free" on the very next `get_effective_tier` call, not at period end as documented. **Note:** `ensure_subscription_tables` hand-writes SQLite-specific `CREATE TABLE IF NOT EXISTS` DDL directly in the service layer rather than using Alembic/ `create_all()` — the same ad hoc schema-management pattern seen in `enterprise.py` , `learn.py` , `business_chat.py` , and `i18n_routes.py` . **Note (dead code — see Chapter 11 for the full picture):** `activate_premium` , `cancel_premium` , `check_idea_limit` , `check_chat_limit` , `increment_idea_count` , `increment_chat_count` , and `get_usage_summary` are defined here but **never called from any route file** in the current codebase — only `activate_trial` , `get_effective_tier` , `get_trial_info` , and `ensure_subscription_tables` are wired in, and only from the unmounted `auth-stripe.py` .

survey_json.py

What this file does: A lightweight file-based (not database-based) persistence layer for market-survey submissions — each Zambian town gets its own JSON file under `backend/data/surveys/<slug>.json` accumulating every submission plus running aggregate statistics.

`save_survey_json(...)` (33-94) re-reads and re-writes the **entire** town JSON file and re-aggregates **all** historical submissions from scratch on every single call — no pagination, streaming, or incremental aggregation. For a popular town, this file (and the in-memory list) grows without bound and gets slower with every submission.

Note: `os.makedirs(SURVEY_DIR, exist_ok=True)` runs as a **module-import-time side effect** — importing this module for any reason creates a directory on disk. **Note (race condition):** No file locking around the read-modify-write cycle — concurrent submissions for the same town under FastAPI's async/threaded request handling can race, silently losing one submission's data. **Note:** The numeric/categorical field name lists used by `_aggregate()` are hardcoded and must be kept in sync by hand with whatever fields the survey form actually submits — a new field added elsewhere won't be aggregated unless someone remembers to add its name here too.

zedtranslate_service.py

What this file does: Wraps a third-party HTTP translation API ("ZedTranslate," `api.zedtranslate.com`) to translate UI strings and AI-generated content into five Zambian languages (Bemba, Nyanja, Tonga, Lozi, Tumbuka). This is a **real external HTTP API call** (via `httpx`), not a static dictionary and not a Claude call. All translations are cached in a SQLite `translation_cache` table keyed by a SHA-256 hash of `(source, target, text)` , so each unique string is only ever sent to the external API once. Every failure mode (missing key, timeout, rate limit, any exception) degrades to returning the original untranslated text rather than raising.

Note: No hardcoded secret — `ZEDTRANSLATE_API_KEY` is read from an environment variable with an empty-string default, handled gracefully if absent. **Note:** Every translation failure is silently absorbed into "return the source text" — a caller has no way to distinguish "successfully translated" from "translation failed, showing English" without inspecting logs, which could present confusing untranslated content to end users with no visible error indicator. **Note:** `translate_sync()` (the blocking counterpart used by an offline batch-warming script) has **no cache lookup at all** — using it outside its intended one-off script context would bypass caching entirely and could unnecessarily multiply API cost. **Note:** `translate_batch()` awaits `translate()` one string at a time in a plain loop with no concurrency (`asyncio.gather`) — translating a large dictionary with many cache misses is slow (fully serialized network round-trips).

10. Frontend Architecture

The frontend is a React 18 + Vite single-page app. Unlike the backend chapters, this chapter covers architecture and patterns rather than a line-by-line walkthrough of every page — the goal is that a new engineer understands how routing, auth, API calls, and state are wired together, and can then read any individual page confidently.

Routing & App shell

Provider order (outermost → innermost), `src/main.jsx` then `src/App.jsx` : `React.StrictMode` → `BrowserRouter` → `AuthProvider` (`main.jsx:19-21`) → `App()`, which itself wraps `ThemeProvider` → `TranslationProvider` → `AuthProvider` (again) → `AppRoutes` (`App.jsx:119-122`).

Note: `AuthProvider` is mounted **twice** — once in `main.jsx:21` and again in `App.jsx:121`. React context resolves to the nearest provider, so every `useAuth()` call actually reads the inner instance; the outer one still runs its own redundant `/auth/me` fetch on mount. Harmless but wasteful and confusing for a new reader trying to trace auth state.

Routes (`AppRoutes`, `App.jsx:50-115`): public routes are `/`, `/login`, `/register`, `/verify-email`, `/pricing`, and the legal/about/contact pages. Protected routes (wrapped in `<Protected>`) cover the entire authenticated app surface: `/billing`, `/dashboard`, `/chat` (+ `/chat/:id`), `/news`, `/ideas`, `/templates`, `/survey`, `/enterprise`, `/capital`, `/market`, `/learn`, `/bizsim`, `/startup` (+ chat sub-routes), and `/business/:planId` (+ `/log`, `/survey` sub-routes). A catch-all redirects unknown paths to `/`.

`Protected` (`App.jsx:42-48`) checks only for token **presence** in `localStorage` (not validity, and not `useAuth()`'s `loading` / `user` state) — an expired token still renders protected content briefly, until the first API call gets a 401 and the interceptor force-redirects.

Auth flow

`useAuth.jsx` (6-41): on mount, reads `kip_token` from `localStorage`; if present, calls `GET /auth/me` to populate `user`, clearing the token on failure. `login(token, userData)` synchronously writes the token and sets state; `logout()` clears both. `LoginPage.jsx` deliberately calls `window.location.replace()` after `login()` rather than `navigate()`, specifically because `Protected`'s `localStorage` read would otherwise race the async React state update.

The JWT attaches to every request via the axios request interceptor in `lib/api.js:11-16`, which reads `kip_token` and sets `Authorization: Bearer <token>`, alongside an `X-KIP-Language` header. On a 401 response, `lib/api.js:19-28` wipes the token and hard-redirects to `/login` via `window.location.href`.

Note (token-key inconsistency): `api.js` and `useAuth.jsx` only ever use the key `kip_token`. But `Protected` and `lib/submitLog.js`'s `getToken()` check **three** possible keys (`kip_token`, `token`, `access_token`). `RegisterPage.jsx` writes all three keys plus `kip_user` and manually sets a default axios header — logic `LoginPage.jsx` does not replicate. This split-brain token storage is fragile and worth consolidating onto a single key.

API layer

`lib/api.js` (30 lines) is the shared axios instance used by nearly every page: `baseUrl` from `VITE_API_URL`, defaulting to `/api` (proxied to the backend in dev). Its two interceptors (auth-header attachment, global 401 handling) are described above.

`lib/offlineQueue.js` is an IndexedDB-backed queue (`kip-offline-queue` DB) for daily-log submissions, synced by a Service Worker background-sync tag (`'kip-log-sync'`) when connectivity returns. `lib/submitLog.js` is the single call site pages should use for log submission: it tries a direct `POST /logs/daily` first, and only falls back to the offline queue on a genuine *network* error (no response at all) — a real 4xx/5xx from the server is re-thrown so the caller can show an actual error rather than silently queueing a request the server already rejected.

Note: Offline detection relies on `navigator.onLine`, which only reflects the OS network-adaptor state, not actual API reachability — a device on Wi-Fi with no real internet appears “online” and will attempt (and fail) a direct POST before any queueing logic kicks in.

State & context patterns

`useTheme.jsx` manages dark/light mode via `localStorage.kip_theme` and a `data-theme` attribute on `<html>`. **Note:** Its `try` / `catch` fallback is inconsistent — an unset theme defaults to `'light'` in the success path but `'dark'` if `localStorage` is inaccessible.

Two **separate, overlapping** language mechanisms exist: - `hooks/useLanguage.jsx` — simple module-level `getLanguage()` / `setLanguage()` functions (used by `api.js` 's interceptor directly, since it's not a React hook and works outside components). - `context/TranslationContext.jsx` — a fuller i18n provider with a `t(key, params)` translator, a pre-built-bundle-then-API-then-English fallback chain for loading strings, and its own in-memory cache.

Both read/write the same `localStorage.kip_language` key and both listen for/dispatch the same `kip-language-change` `CustomEvent` , so they stay loosely synchronized — but having two independent implementations of “the current language” is a real risk for a future engineer wiring up a new language-aware feature and picking the wrong one.

Page inventory

File	Route(s)	Purpose
LandingPage.jsx	<code>/</code>	Public marketing page.
LoginPage.jsx	<code>/login</code>	Login form → <code>useAuth().login()</code> .
RegisterPage.jsx	<code>/register</code>	Signup form; writes all 3 legacy token keys.
VerifyEmailPage.jsx	<code>/verify-email</code>	Consumes <code>?token=</code> , calls <code>/auth/verify-email</code> .
DashboardPage.jsx	<code>/dashboard</code>	Authenticated home — stats, charts, quick links.
ChatPage.jsx	<code>/chat</code> , <code>/chat/:id</code>	General AI chat (K-BIG-1).
NewsPage.jsx	<code>/news</code>	Economic news/briefings feed.
IdeasPage.jsx	<code>/ideas</code>	Browse/manage AI-generated ideas; “add your own business” flow.
BusinessDashboardPage.jsx	<code>/business/:planId</code>	Per-business dashboard with AI chat + plan-update approvals.
EnhancedLogPage.jsx	<code>/business/:planId/log</code>	Daily log entry form; uses <code>submitLog()</code> for offline support.
SurveyPage.jsx	<code>/business/:planId/survey</code>	Business-specific market survey.
TemplatesPage.jsx	<code>/templates</code>	Forms/letters library + AI letter generation.
GeneralSurveyPage.jsx	<code>/survey</code>	Standalone market survey submission.
EnterprisePage.jsx	<code>/enterprise</code>	Multi-branch management, gated behind a <code>PremiumGate</code> .
legal/TermsPage.jsx, PrivacyPage.jsx, OtherPages.jsx	<code>/legal/*</code> , <code>/about</code> , <code>/contact</code>	Static legal/info content.
CapitalAccessPage.jsx	<code>/capital</code>	Funding provider list (CEEC, CDF, DBZ, ZANACO).
MarketIntelligencePage.jsx	<code>/market</code>	Aggregated market survey data by town.
LearnPage.jsx	<code>/learn</code>	Courses, badges, progress.
BizSimPage.jsx	<code>/bizsim</code>	The business simulation game (recharts-based).
StartupPage.jsx	<code>/startup</code>	Registration/compliance roadmap (PACRA, ZRA, NAPSA).
StartupChatPage.jsx	<code>/startup/chat</code> , <code>/startup/chat/:stepId</code>	Dedicated startup-registration AI chat.
PricingAndBilling.jsx	<code>/pricing</code> , <code>/billing</code>	Exports both <code>PricingPage</code> (public) and <code>BillingPage</code> (protected).
SettingsPage.jsx	<i>not routed</i>	Full settings UI on disk but not imported in <code>App.jsx</code> — orphaned.

File	Route(s)	Purpose
PricingPage.jsx	<i>not routed</i>	A second, unused <code>PricingPage</code> implementation — <code>App.jsx</code> uses the one in <code>PricingAndBilling.jsx</code> instead.
AuthPages-l2.jsx	<i>not routed</i>	Alternate Login/Register implementations, not imported anywhere.
RegisterPage.jsx.txt	n/a	Not a <code>.jsx</code> file — excluded from the build; a stray backup.

Component inventory

File	Purpose
Layout.jsx	Authenticated app shell (sidebar, mobile drawer, bottom nav); polls <code>/news/alerts/count</code> every 5 min.
AddBusinessModal.jsx	“Add existing business” modal → <code>POST /business/add-own</code> .
KipMarkdown.jsx	Custom lightweight Markdown renderer for AI chat responses.
LanguageToggle.jsx	UI language switcher.
OfflineBanner.jsx	Shown when offline / logs pending sync.
TrialBanner.jsx	Shows remaining free-trial days (polls <code>/stripe/status</code> — see Chapter 11 for why this currently 404s).
UpgradeModal.jsx	Premium upgrade modal (Stripe + mobile money) — see Chapter 11.
bizsim/StorySetup.jsx, EventCard.jsx, GameAudio.jsx, DayCanvas.jsx, theme.js	The BizSim game’s dedicated UI layer.
Footer.jsx, KipBackground.jsx, KipIllustrations.jsx, ParticleBackground.jsx, LegalLayout.jsx	Shared presentational/branding components.

Notable patterns and issues

- **Duplicate** `AuthProvider` in the provider tree (see above) — redundant `/auth/me` fetch on every load.
- **Protected gates on token presence only**, not validity — brief flash of protected UI on an expired session.
- **Three-way token-key inconsistency** across `api.js` / `useAuth.jsx` (`kip_token` only) vs. `Protected` / `submitLog.js` (checks all three) vs. `RegisterPage.jsx` (writes all three) — a latent source of bugs if any one key is cleared inconsistently.
- **Two parallel language systems** (`useLanguage.jsx` and `TranslationContext.jsx`) doing overlapping jobs.
- **Dead page files** on disk but not routed: `SettingsPage.jsx` , a duplicate `PricingPage.jsx` , and `AuthPages-l2.jsx` — these can confuse a future engineer searching for “the” implementation of settings/pricing/login.
- `navigator.onLine` -based **offline detection** only reflects adapter state, not real connectivity.

11. Challenges, Vulnerabilities & Solutions

This chapter is a security and technical-debt audit performed by reading the actual source (not a scanner report). Every finding below was verified against the code as of commit `cc47629` (17 July 2026). Findings are grouped by topic; each includes severity, evidence (`file:line`), and a concrete recommended fix.

The headline risk, in one paragraph: the most recent commit deleted the entire Stripe/Flutterwave payment backend (`stripe_routes.py` , `flutterwave_routes.py`) without removing the frontend's calls to it or wiring the pre-existing usage-limit enforcement code into any AI-generating endpoint. The practical result is two compounding problems: (1) payments are currently non-functional while the billing UI still tries to call them, and (2) **every Claude-calling endpoint in the app has zero cost control** — the rate-limiting logic exists in `subscription_service.py` but is not called from anywhere. Fixing the wiring on both of these should be the top priority before this system is exposed to real traffic again.

1. Secrets management

File:Line	Severity	Finding	Solution
<code>backend/app/security.py:12</code>	High	<code>SECRET_KEY</code> silently falls back to a hardcoded, publicly-known default (<code>"kip-dev-secret-change-in-production"</code>) if the env var is unset. If deployed without it set, anyone can forge valid JWTs for any user ID.	Fail fast at startup: <code>if not os.getenv("SECRET_KEY"): raise RuntimeError(...)</code> outside of an explicit local-dev mode. Remove the insecure default entirely.
<code>.gitignore</code>	Informational (good)	<code>.env</code> / <code>.env.*</code> are correctly gitignored; only <code>.env.example</code> (placeholder values only) is tracked.	No action needed.
<code>backend/Dockerfile</code> , <code>backend/fly.toml</code> , <code>frontend/vercel.json</code>	Informational	No embedded secrets found; <code>fly.toml</code> only sets non-sensitive env vars.	Confirm real secrets are injected via <code>fly secrets set</code> / Vercel's secret manager, not baked into images.
Codebase-wide grep for hardcoded keys/passwords	Informational (good)	None found. <code>ANTHROPIC_API_KEY</code> is loaded via <code>os.getenv(...)</code> consistently across 15+ files.	The repeated <code>os.getenv("ANTHROPIC_API_KEY", "")</code> pattern in 15+ files is a maintainability smell — worth centralizing in one config module — but not a security issue.

2. Authentication & JWT

File:Line	Severity	Finding	Solution
<code>backend/app/security.py:13-14</code>	Informational	HS256, 7-day expiry — a reasonable but long-lived bearer token.	Consider shortening, or pairing with a refresh-token flow, if risk tolerance requires it.
<code>backend/app/security.py</code> (whole file)	Medium	No token revocation/blocklist and no <code>/logout</code> route exists anywhere in the codebase. A JWT is valid for 7 days no matter what — password change or account compromise doesn't invalidate it.	Add a <code>token_version</code> or <code>password_changed_at</code> column on <code>User</code> , embed it in the JWT payload, and check it in <code>get_current_user</code> ; bump it on password change to invalidate old tokens.

File:Line	Severity	Finding	Solution
<code>backend/app/routes/*.py</code>	Informational (good)	<code>Depends(get_current_user)</code> is used consistently on every real endpoint. <code>bizsim_engine.py</code> / <code>language_support.py</code> initially looked like exceptions but define no <code>@router</code> endpoints themselves — they're logic modules imported by routers that do enforce auth.	No action needed.
<code>backend/app/schemas.py:6-13</code>	Medium	<code>UserRegister.password</code> has no <code>min_length</code> /complexity/max-length validator — a 1-character password is accepted.	Add a Pydantic validator enforcing a sane minimum (8+) and maximum length (bcrypt silently truncates input beyond 72 bytes).
<code>backend/app/routes/auth.py</code> (<code>/register</code> , <code>/login</code>)	High	No rate limiting anywhere in the codebase (see §7) — login is brute-forceable, registration is spammable/enumerable.	Add IP- and email-based rate limiting (e.g. <code>slowapi</code>) on <code>/api/auth/login</code> and <code>/api/auth/register</code> . The existing generic "incorrect email or password" message is already good practice — keep it.
<code>backend/app/routes/auth-stripe.py:48,88</code>	Medium (dormant bug)	<code>create_access_token({"sub": str(user.id)})</code> passes a dict where <code>security.py</code> expects an <code>int</code> — every token minted by this file would be permanently unusable. Not currently exploitable since this router isn't mounted (see §5, §10).	Delete this file (superseded by <code>auth.py</code>), or fix the call signature if the trial-on-register behavior is ever restored.

3. CORS

File:Line	Severity	Finding	Solution
<code>backend/main.py:77-89</code>	Informational (good)	<code>allow_origins</code> is an explicit allowlist (two localhost origins + <code>FRONTEND_URL</code>), not <code>["*"]</code> . <code>allow_credentials=True</code> combined with this explicit list is safe — the classic wildcard+credentials misconfiguration does not apply here.	Confirm <code>FRONTEND_URL</code> is set to the exact production origin in Fly secrets; if unset, only the two localhost origins remain allowed, meaning a misconfigured deploy fails closed (blocks the real frontend) rather than open.

4. Injection risks

File:Line	Severity	Finding	Solution
Every <code>db.execute(text(...))</code> site across <code>bizsim_routes.py</code> , <code>business_chat.py</code> , <code>learn.py</code> , <code>templates.py</code> , <code>enterprise.py</code> , <code>i18n_routes.py</code> , <code>subscription_service.py</code> , <code>zedtranslate_service.py</code>	Informational (good)	Every raw-SQL call site uses SQLAlchemy bound parameters, not string concatenation of user input. No SQL injection vector was found anywhere in this review.	No action needed.
<code>backend/app/routes/bizsim_routes.py:108</code>	Low	One f-string-built <code>ALTER TABLE</code> statement interpolates a column name/type — but only from a hardcoded internal migration list, never from request input.	Defensive habit only: whitelist-validate even internally-sourced identifiers, in case this code is ever repurposed for dynamic input.

File:Line	Severity	Finding	Solution
<code>backend/app/routes/templates.py:431-437</code> (<code>GET /admin/towns/{slug}</code>)	Medium (path traversal, not SQL)	The <code>slug</code> path parameter is concatenated directly into a filesystem path with no sanitization, unlike the write path (<code>_slug()</code>), which does sanitize.	Sanitize <code>slug</code> the same way <code>_slug()</code> does (or reject non-alphanumeric characters) before building the file path.

5. Payment security

File:Line	Severity	Finding	Solution
<code>frontend/src/pages/PricingAndBilling.jsx</code> calls <code>/api/flutterwave/verify/{txRef}</code> , <code>/api/stripe/status</code> , <code>/api/stripe/billing-portal</code>	High	No Stripe or Flutterwave router is registered in <code>backend/main.py</code> . Commit <code>cc47629</code> deleted <code>stripe_routes.py</code> (327 lines) and <code>flutterwave_routes.py</code> (270 lines) — confirmed via <code>git show cc47629 --stat</code> — and stale compiled <code>.pyc</code> artifacts for both still sit in <code>__pycache__</code> . The entire payment backend is currently non-functional while the frontend still calls it.	Either restore and re-register both route files, or gate the pricing/billing UI behind a feature flag until payments are reworked, so it doesn't ship broken to users.
Prior revision of <code>stripe_routes.py</code> (no longer in the working tree)	Medium (historical)	The deleted version implemented <code>stripe.Webhook.construct_event(...)</code> signature verification, but only when <code>STRIPE_WEBHOOK_SECRET</code> was set — otherwise it fell back to parsing the payload unverified ("dev only").	When payment routes are restored, make signature verification mandatory in any non-development environment — fail closed, not "dev only" fallback.
<code>backend/app/services/subscription_service.py:197-241</code> (<code>activate_premium</code>)	Medium	Computes <code>amount_zmw</code> server-side (good), but is never called from any route — confirmed via grep. There is currently no reachable path to actually becoming premium except the automatic 30-day trial in the unmounted <code>auth-stripe.py</code> .	Once payment routes are restored, ensure only the webhook handler calls <code>activate_premium</code> — never a client-supplied field.
<code>backend/requirements.txt</code>	Low	Still lists <code>stripe>=7.0.0</code> even though nothing imports it anymore.	Remove the orphaned dependency, or restore the code that uses it.

6. File uploads / path traversal

File:Line	Severity	Finding	Solution
Whole backend	Informational (good)	No <code>UploadFile</code> usage anywhere — there is no file-upload feature to review.	No action needed.
<code>backend/app/routes/templates.py:31-35</code> (<code>_slug()</code>) and <code>backend/app/services/survey_json.py:19-20, 57</code>	Informational (good)	Strips input to <code>[a-z0-9\s]</code> before building a per-town cache filename — path traversal via a submitted <code>location</code> string is not possible on the write path.	As defense-in-depth, also validate the resulting path stays under the intended directory via <code>os.path.realpath</code> + prefix check.
<code>backend/app/routes/templates.py:431-437</code>	See §4 above — the read path (<code>GET /admin/towns/{slug}</code>) does not apply the same sanitization as the write path.	See §4.	

7. Rate limiting & Claude API cost control

File:Line	Severity	Finding	Solution
Entire backend	Critical	No rate-limiting middleware or library exists anywhere in the codebase.	Add IP- and user-based rate limiting (e.g. <code>slowapi</code> + Redis token bucket) in front of every endpoint, at minimum the AI-generation ones.
<code>backend/app/services/subscription_service.py:115-173</code>	Critical	A complete daily-usage-limit system exists (3 ideas/day, 5 chats/day for free tier) but <code>check_idea_limit</code> , <code>check_chat_limit</code> , <code>increment_idea_count</code> , and <code>increment_chat_count</code> are never imported or called by any route — confirmed via <code>grep</code> . Every Claude-calling endpoint (15+ files instantiate <code>anthropic.Anthropic(...)</code> , including <code>chat.py</code> , <code>business_chat.py</code> , <code>bizsim_routes.py</code> , <code>enterprise.py</code>) runs with zero usage caps for any authenticated user.	Wire <code>check_idea_limit</code> / <code>check_chat_limit</code> into every Claude-calling endpoint before the API call, and the matching <code>increment_*</code> after a successful generation. This is the single highest-priority fix for cost control.

8. Dependencies & deployment

File:Line	Severity	Finding	Solution
<code>backend/requirements.txt</code>	Medium	Nearly all dependencies pinned with <code>>=</code> rather than exact versions (<code>chromadb</code> , <code>sentence-transformers</code> , <code>httpx</code> , <code>anthropic</code> , etc.) — only <code>fastapi</code> / <code>uvicorn</code> / <code>sqlalchemy</code> use <code>==</code> . Every fresh install can silently pull untested major versions.	Pin exact versions (or use a lockfile via <code>pip-compile</code>) and bump deliberately with testing.
<code>backend/Dockerfile</code>	Medium	No <code>USER</code> directive — the container runs as root in <code>python:3.13-slim</code> . Single worker, no restart-scaling concern noted beyond that.	Add a non-root user (<code>RUN useradd -m appuser + USER appuser</code>) before <code>CMD</code> .
<code>backend/fly.toml</code>	Informational (good)	Explicit health check + <code>force_https = true</code> .	No action needed.
<code>frontend/package.json</code>	Low	Dependency versions (React 18.3, Vite 5.2, axios 1.7.2) are reasonably current; no live CVE lookup performed.	Run <code>npm audit</code> / <code>pip-audit</code> periodically — no CI currently configured for this per CLAUDE.md.

9. Cross-site scripting (XSS)

File:Line	Severity	Finding	Solution
<code>frontend/src/components/KipMarkdown.jsx</code>	Informational (good)	Renders AI-generated markdown into plain React elements with string children — no <code>dangerouslySetInnerHTML</code> . React auto-escapes string content, so raw HTML/script in Claude's output renders as inert text.	No action needed.
Entire <code>frontend/src</code> tree	Informational (good)	Zero matches for <code>dangerouslySetInnerHTML</code> anywhere in the frontend.	No action needed — worth documenting as a clean result.

10. Untracked / backup files

File	Status	Notes
<code>backend/app/routes/auth-stripe.py</code>	Not registered in <code>main.py</code> — dead code containing the <code>create_access_token(dict)</code> bug from §2.	Delete, or merge the trial-on-register behavior into <code>auth.py</code> if still desired, then remove this file.
<code>backend/app/routes/stripe_routes.py</code> , <code>flutterwave_routes.py</code>	Deleted in commit <code>cc47629</code> ; stale <code>.pyc</code> artifacts remain; frontend still calls their endpoints (§5).	Restore and re-register, or strip the frontend calls to match current backend state; clear stale <code>__pycache__</code> in deploy tooling.
<code>backend/app/schemas-b4-stripe2.py</code> , <code>schemas-stripe.py</code> , <code>main-stripe.py</code>	Already deleted in the same commit — confirmed gone from the working tree.	No action needed.
<code>frontend/public/sw-b4-offline.js</code> (+ its <code>dist/</code> build copy)	Still present, matches the repo's own <code>-b4-</code> backup-file convention (see CLAUDE.md). Service workers are risky to leave as dead code since browsers persist them across sessions.	Confirm it's not registered anywhere (<code>navigator.serviceWorker.register</code>), then delete if genuinely a backup.
<code>nul</code> (repo root)	A 0-byte Windows shell-redirect artifact.	Delete and add <code>nul</code> to <code>.gitignore</code> .
<code>backend/kip_logo_v2.png</code>	A versioned-suffix asset — matches the "old branch kept in tree" pattern CLAUDE.md warns about.	Confirm which logo file is actually referenced (<code>email_service.py</code> resolves its own path relative to itself) and remove the unused one.

Overall risk ranking: the two most consequential findings are (1) **no cost control on any Claude-calling endpoint** (§7 — the enforcement code exists but is wired to nothing) and (2) **payment processing is currently entirely absent from the backend while the frontend still calls it** (§5) — both stem from the same recent commit that deleted the payment routers without removing their frontend call sites or wiring the pre-existing subscription limits into any AI-generating route. The hardcoded `SECRET_KEY` fallback (§1) is the most severe *pure* auth-security bug, since it becomes exploitable the moment `SECRET_KEY` is left unset in any environment.

Appendix A — Consolidated Punch List

A prioritized, single-page view of every issue raised across Chapters 2–11, for triage/ticket creation. “Where” gives the primary file; see the relevant chapter for full evidence and line numbers.

Do first (security / cost / broken-in-production)

1. **No cost control on any Claude-calling endpoint.** `subscription_service.py`’s usage-limit functions exist but are called from nowhere. (Ch. 9, 11 §7)
2. **Payment backend deleted, frontend still calls it.** `stripe_routes.py` / `flutterwave_routes.py` removed in the latest commit; `PricingAndBilling.jsx` still hits `/api/stripe/*` and `/api/flutterwave/*`. (Ch. 11 §5)
3. **SECRET_KEY has an insecure hardcoded fallback.** `security.py:12`. (Ch. 2, 11 §1)
4. **No rate limiting anywhere** — login brute-force, registration spam, and expensive AI endpoints are all unprotected. (Ch. 11 §7)
5. `enhanced_logs.py` `/survey` **update path throws `NameError` on every call** (`survey_obj` undefined). (Ch. 5)
6. `bizsim_routes.py` `/day/run` **cash-inflation exploit** via negative `stock_buy` / `marketing` / `save_amount`, plus a possible unhandled 500 from negative `marketing`. (Ch. 7)
7. `log_service.py` **weekly-review fallback crashes with `NameError`** (`kips` vs `kpis` typo), uncaught by any surrounding handler. (Ch. 9)

Do soon (correctness bugs, data integrity)

8. `auth-stripe.py` mints permanently-broken tokens (`create_access_token` called with a dict) — currently harmless only because the router isn’t mounted. (Ch. 4, 11 §2)
9. `subscription_service.cancel_premium` downgrades a user to free **immediately**, not at period end as documented. (Ch. 9)
10. BizSim’s loan-interest ledger posting double-counts cash and desyncs `loan_payable` from the session’s real `loan_balance` — the balance sheet and session state will visibly disagree over a long game. (Ch. 7)
11. BizSim’s narrative debt (repay by day 30 or consequences) is never mechanically enforced — no deduction, no due-day check, no `/debt/repay` endpoint. (Ch. 7)
12. `dashboard_service.check_profitability` has an unreachable branch due to a threshold mismatch (`>= 30` vs `>= 28`). (Ch. 9)
13. `news_service.fetch_fuel_prices()` scrapes a Zimbabwean domain (`zera.co.zw`) instead of Zambia’s regulator. (Ch. 8)
14. `templates.py` `GET /admin/towns/{slug}` — path-traversal-shaped input is not sanitized on the read path (write path is). (Ch. 6, 11 §4)
15. `templates.py` `/admin/towns*` endpoints have no actual admin/role check despite the path name. (Ch. 6, 11 §2)
16. `learn.py` leaks every lesson quiz’s answer key in the course-detail response, before the user attempts it. (Ch. 7)
17. `learn.py` `/submit-assessment` can raise an unhandled `IndexError` (500) on an out-of-range answer index. (Ch. 7)
18. `news.py` `GET /alerts` docstring says “unread” but returns all alerts (no `is_read` filter), unlike its sibling `/alerts/count`. (Ch. 6)
19. `news.py` `/rates/refresh` and `/articles/refresh` have **no authentication at all**. (Ch. 6)
20. `enterprise.py`’s premium gate (`_require_premium`) is commented out at its only call site. (Ch. 6)

Worth scheduling (tech debt, consistency, performance)

21. Duplicate daily-log schemas: `DailyBusinessLog` vs. `EnhancedDailyLog` both exist and both write similar data. (Ch. 3)
22. Duplicate chat-thread schemas: `Conversation` / `Message`, `StartupConversation` / `StartupMessage`, and `business_chat_model.py`’s tables are three near-identical designs. (Ch. 3)
23. `User` model has inconsistent relationship coverage — only 2 of ~10 tables that FK into `users.id` have a `relationship()` back to `User`. (Ch. 3)
24. Most FK columns across the schema (`plan_id`, `user_id` on many tables) are not indexed despite being the dominant query pattern. (Ch. 3)
25. Several files hand-roll their own SQLite-only `CREATE TABLE IF NOT EXISTS` migrations at request time instead of using a real migration tool (`business_chat.py`, `enterprise.py`, `learn.py`, `subscription_service.py`, `zedtranslate_service.py`, `i18n_routes.py`) — none of this will work if the app is ever pointed at Postgres via `DATABASE_URL`. (Ch. 2, 5–9)
26. The Claude model ID `"claude-sonnet-4-6"` is hardcoded as a repeated string literal in well over a dozen call sites across the codebase rather than a single shared constant/env var. (Ch. 8–9)

27. No prompt-caching (`cache_control`) applied anywhere, despite `kip_prompt.py` 's multi-hundred-token static system prompt being byte-identical on every request. (Ch. 8)
28. Several `async def` functions make blocking, synchronous Anthropic SDK calls directly (`bizzsim_engine.py` 's three AI functions, `kip_engine.py`), stalling the FastAPI event loop for the duration of each API round-trip. (Ch. 7, 8)
29. N+1 query patterns in `dashboard.py` (`/recent-ideas`), `enterprise.py` (branch/manager lookups), and `news.py` (per-article, per-user alert generation). (Ch. 4, 6)
30. Several endpoints return unbounded result sets with no pagination cap (`chat.py` / `ideas.py` / `startup_chat.py` conversation lists, `business_dashboard.py` `/logs/{plan_id}` , `news.py` `/articles` `limit` / `offset`). (Ch. 4–6)
31. Two parallel frontend language systems (`useLanguage.jsx` and `TranslationContext.jsx`) doing overlapping jobs. (Ch. 10)
32. Three-way inconsistent `localStorage` token-key usage across the frontend (`kip_token` vs. checking all of `kip_token` / `token` / `access_token`). (Ch. 10)
33. Dead/orphaned frontend files: `SettingsPage.jsx` , a duplicate `PricingPage.jsx` , `AuthPages-l2.jsx` , `RegisterPage.jsx.txt` . (Ch. 10)
34. Inconsistent error-logging style across services — some use `print()` , one uses `logging` (`briefing_service.py`) — no centralized structured logging. (Ch. 8–9)

Appendix B — Glossary

API key — A secret string that authenticates KIP's backend to an external service (here, mainly Anthropic's Claude API). Must never be committed to source control; KIP reads it from the `ANTHROPIC_API_KEY` environment variable.

Bearer token — An access credential sent in an HTTP `Authorization: Bearer <token>` header. KIP uses a JWT as its bearer token.

bcrypt — A deliberately slow, salted password-hashing algorithm. KIP uses it (`app/security.py`) so that even if the database is leaked, recovering plaintext passwords is computationally expensive.

CORS (Cross-Origin Resource Sharing) — A browser security mechanism that blocks a web page from calling an API on a different origin (domain/port) unless that API explicitly allows it. KIP's backend allowlists its known frontend origins in `main.py` .

Dependency injection (FastAPI) — FastAPI's pattern where a route function declares parameters like `db: Session = Depends(get_db)` ; FastAPI automatically calls `get_db()` and supplies the result before running the route body. Used throughout KIP for database sessions and the logged-in user.

Embedding — A numeric vector representation of a piece of text, generated by a model such as `sentence-transformers/all-MiniLM-L6-v2` , such that texts with similar meaning have vectors that are close together. This is what makes semantic search (as opposed to keyword search) possible.

JWT (JSON Web Token) — A signed, self-contained token format used for stateless authentication. KIP's JWTs contain the user's ID and an expiry timestamp, signed with `SECRET_KEY` using the HS256 algorithm. "Stateless" means the server doesn't need to store session data — it just verifies the signature on each request.

N+1 query — A performance anti-pattern where code runs one query to fetch a list of N items, then loops over them running one additional query per item (N more queries) instead of fetching everything in a single, larger query.

ORM (Object-Relational Mapper) — A library (here, SQLAlchemy) that lets Python code work with database rows as Python objects (`User` , `Conversation` , etc.) instead of writing raw SQL for every operation.

Pydantic schema — A Python class (subclassing `BaseModel`) that defines the exact shape of data FastAPI expects in a request or will return in a response. FastAPI uses these to validate incoming JSON automatically and to control exactly which fields get serialized into a response (so, for example, a password hash never accidentally leaks into an API response).

RAG (Retrieval-Augmented Generation) — A technique where, before asking an AI model a question, the system first searches a knowledge base for relevant information and includes it in the prompt. KIP does this via ChromaDB (`knowledge_base.py`) to ground its advice in curated Zambian business/economics content rather than relying solely on what the model already "knows."

Router (FastAPI `APIRouter`) — A FastAPI object that groups related endpoints together (e.g. all authentication endpoints). `main.py` "mounts" each router under a URL prefix like `/api/auth` .

SQL injection — A vulnerability where untrusted input is concatenated directly into a SQL query string, letting an attacker manipulate the query. KIP avoids this throughout by using parameterized queries (`:param` placeholders bound to values) rather than string formatting — confirmed clean in this review (Chapter 11 §4).

Webhook — An HTTP callback a third-party service (like Stripe) sends to your backend to notify it of an event (e.g. "payment succeeded"). Verifying the webhook's cryptographic signature is essential so an attacker can't forge a fake "payment succeeded" call.